Ostrovsky_Eliana_Notebook_TP3

May 11, 2025

0.1 Trabajo Práctico: Redes Neuronales

El objetivo de este trabajo es desarrollar y evaluar modelos basados en redes neuronales, incorporando técnicas de ablación para entender el impacto de diversas modificaciones en el proceso de entrenamiento y en la capacidad de generalización del modelo. No se permite usar librerías de machine learning como scikit-learn o PyTorch, a menos que sea pedido explícitamente en el enunciado del ejercicio.

[20]:
```
%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

0.1.1 1. Análisis y Preprocesamiento de Datos

- El dataset que vamos a utilizar es similar a MNIST pero con caracteres japoneses.
- Son imágenes de 28×28 con 49 clases posibles.
- Examinar el dataset y visualizar al menos 3 imágenes.
- Dividir el conjunto de datos en tres subconjuntos: Train, Validation y Test.
- Dividir todos los valores por 255, para que el máximo sea 1.

[21]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from src.preprocessing import *
from src.plots import *
from src.neural_network import *
from src.improved_neural_network import *
from src.experiment import *
from src.utils import *

import torch
from src.pytorch_network import PyTorchNetwork
```

[3]:
```
X_images = np.load("data/X_images.npy")
y_images = np.load("data/y_images.npy")
```

```
print("Forma de X_images:", X_images.shape)
print("Forma de y_images:", y_images.shape)
print("Número de clases únicas:", len(np.unique(y_images)))
```

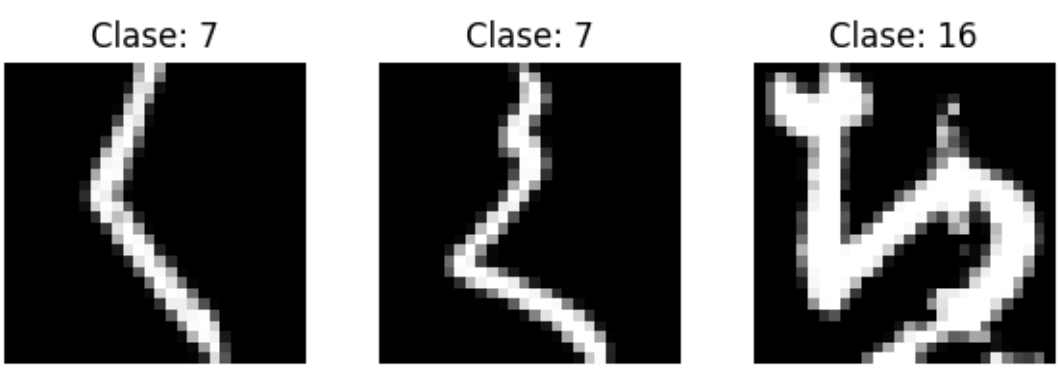
```
Forma de X_images: (5000, 784)
Forma de y_images: (5000,)
Número de clases únicas: 49
```

1. Una visión clara de la distribución de clases para detectar posibles desbalances
2. Estadísticas sobre los valores de píxeles para entender las propiedades generales de las imágenes
3. Imágenes "promedio" de cada clase para visualizar sus características distintivas
4. Un análisis de complejidad por clase para ver cuáles pueden ser más difíciles de clasificar
5. Múltiples ejemplos de cada clase para comprender mejor la variabilidad dentro de cada categoría

[ ]:
```
visualize_samples(X_images, y_images, num_samples=5, num_classes=10,␣
↪figsize=(8, 15))
plot_class_distribution(y_images)
```

[ ]:
```
plot_class_averages(X_images, y_images, num_classes=49)
```

[6]:
```
plt.figure(figsize=(6, 2))
for i in range(3):
    plt.subplot(1, 3, i+1)
    idx = np.random.randint(0, len(X_images))
    img = X_images[idx].reshape(28, 28)
    plt.imshow(img, cmap='gray')
    plt.title(f"Clase: {y_images[idx]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



[6]:
```
X_train, X_val, X_test, y_train, y_val, y_test = stratified_split(
    X_images / 255.0 , y_images, train_ratio=0.8 * 0.8, val_ratio=0.8 * 0.2,␣
 ↪test_ratio=0.2, random_state=42
)

print(f"X_train: {X_train.shape}, y_train: {len(y_train)}")
print(f"X_val: {X_val.shape}, y_val: {len(y_val)}")
print(f"X_test: {X_test.shape}, y_test: {len(y_test)}")
print(f"Rango de valores en X_train: [{X_train.min()}, {X_train.max()}]")
```

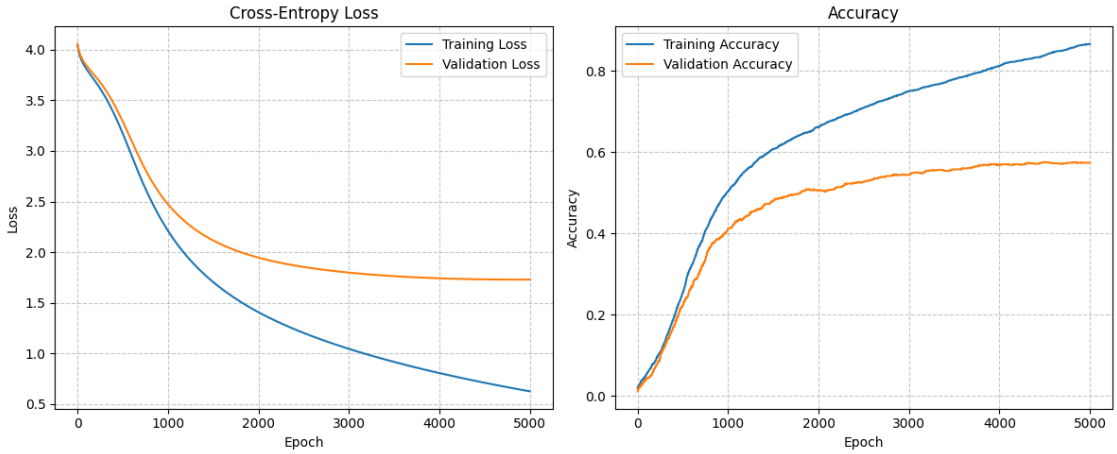
```
X_train: (3197, 784), y_train: 3197
X_val: (802, 784), y_val: 802
X_test: (1001, 784), y_test: 1001
Rango de valores en X_train: [0.0, 1.0]
```

0.1.2 2. Implementación y Entrenamiento de una Red Neuronal Básica

- Implementar una red neuronal con L capas ocultas cada con M (l) nodos confunción de activación ReLU (en las capas ocultas) y activación softmax para la capa de salida.
- Implementar un algoritmo para entrenar dicha red, mediante backpropagation y gradiente descendiente estándar, utilizando como función de costo la cross-entropy. NOTA: El algoritmo backpropagation debe sera adaptado para el caso de clasificación multi-clase con función de activación softmax para la capa de salida y función de costo cross-entropy.
- Entrenar una red neuronal con 2 capas ocultas, con 100 y 80 nodos respectivamente, y graficar la evolución de la función de costo (cross-entropy) sobre los conjuntos de entrenamiento y validación a lo largo de las épocas. Llamaremos a este modelo M0. Reportar las siguientes métricas de performance, sobre los conjuntos de entrenamiento y validación, para el modelo base entrenado:
  - Accuracy
  - Cross-Entropy
  - Matriz de Confusión

[7]:
```
input_size = X_train.shape[1]
output_size = len(np.unique(y_train))
layer_sizes = [input_size, 100, 80, output_size]

model_M0 = NeuralNetwork(layer_sizes, learning_rate=0.01)

history_M0 = model_M0.train(X_train, y_train, X_val, y_val, epochs=5000)
save_model(model_M0, "model_M0.pkl")

plot_learning_curves(history_M0)
```

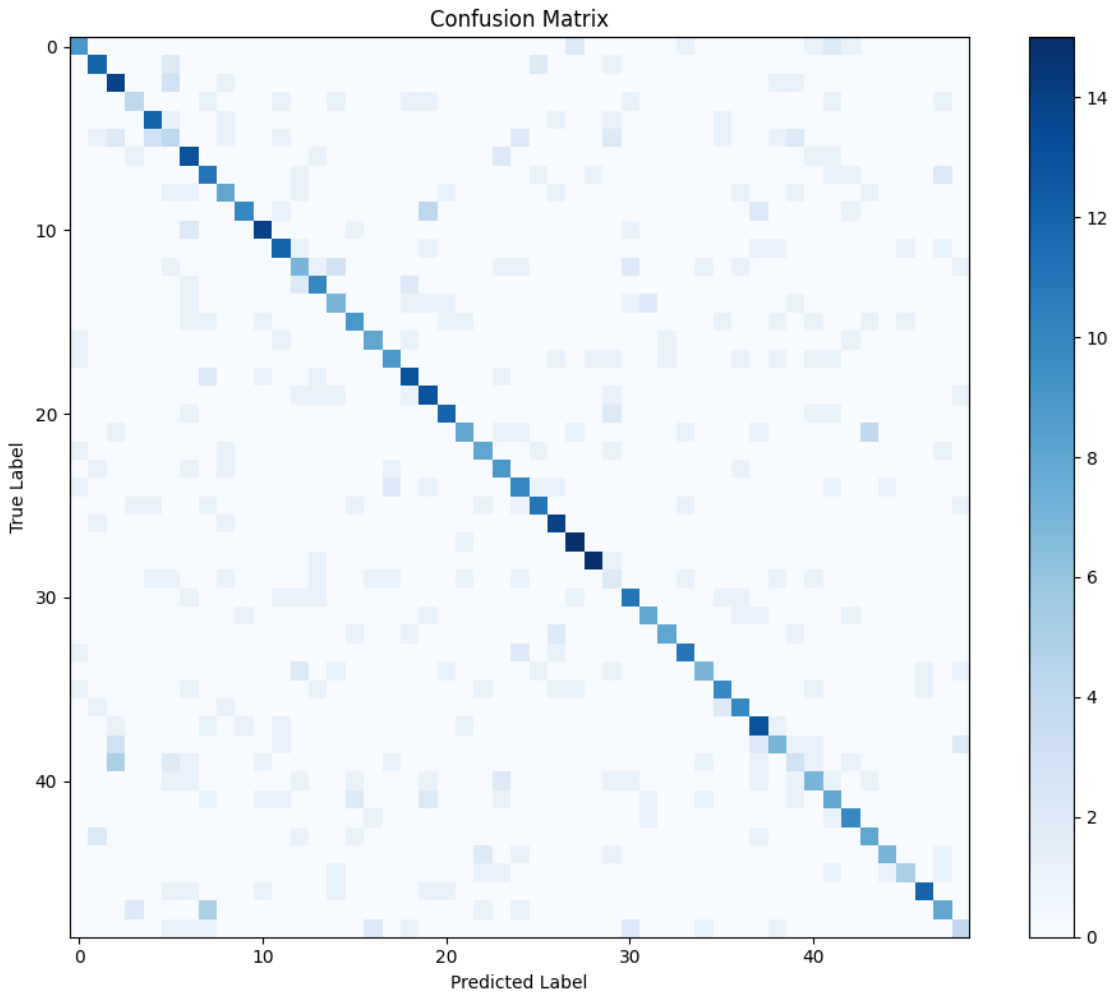
```
Epoch: [==================================================>] 100% - train_loss:
0.6248 - train_acc: 0.8661 - val_loss: 1.7288 - val_acc: 0.5736
Model successfully saved to modelos/model_M0.pkl
```



[8]:
```
train_accuracy, train_loss, train_pred = model_M0.evaluate(X_train, y_train)
val_accuracy, val_loss, val_pred = model_M0.evaluate(X_val, y_val)

print(f"Performance Metrics for Model M0:")
print(f"Training Set - Accuracy: {train_accuracy:.4f}, Cross-Entropy:␣
↪{train_loss:.4f}")
print(f"Validation Set - Accuracy: {val_accuracy:.4f}, Cross-Entropy: {val_loss:
↪.4f}")
```

```
Performance Metrics for Model M0:
Training Set - Accuracy: 0.8661, Cross-Entropy: 0.6248
Validation Set - Accuracy: 0.5736, Cross-Entropy: 1.7288
```

[12]:
```
conf_matrix = model_M0.confusion_matrix(y_val, val_pred)
plot_confusion_matrix(conf_matrix)
```



0.1.3 3. Implementación y Entrenamiento de una Red Neuronal Avanzada

- Implementar las siguientes mejoras al algoritmo de entrenamiento, y para cada una reportar el efecto observado sobre el tiempo de entrenamiento y la performance del modelo resultante.
  - Rate scheduling lineal (con saturación) y exponencial.
  - Mini-batch stochastic gradient descent.
  - Optimizador ADAM.
  - Regularización L2.
  - Regularización mediante early stopping.
  - OPCIONAL: Regularización mediante dropout.
  - OPCIONAL: batch normalization.
- Explorar cambios en la arquitectura de la red (es decir, la cantidad de capas ocultas y unidades ocultas por capa), y los hiperparámetros (cada uno de los items en la lista anterior tiene una serie de parámetros que podemos variar), y determinar la configuración que funcione mejor (menor error de validación). Llamaremos a este modelo M1.

[13]:
```
input_size = X_train.shape[1]
output_size = len(np.unique(y_train))
layer_sizes = [input_size, 100, 80, output_size]
```

[14]:
```
experiments = [
    {
        'name': 'Baseline (SGD)',
        'learning_rate': 0.01,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 2000,
        'batch_size': None,
        'optimizer': 'sgd',
        'lr_schedule': None,
        'early_stopping_patience': None
    }, {
        'name': 'Linear Rate Scheduling',
        'learning_rate': 0.01,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 2000,
        'batch_size': None,
        'optimizer': 'sgd',
        'lr_schedule': 'linear',
        'early_stopping_patience': None
    },{
        'name': 'Exponential Rate Scheduling',
        'learning_rate': 0.01,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 2000,
        'batch_size': None,
        'optimizer': 'sgd',
        'lr_schedule': 'exponential',
        'early_stopping_patience': None
    },{
        'name': 'Mini-batch SGD',
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 2000,
        'batch_size': 64,
        'optimizer': 'sgd',
```

```
        'lr_schedule': None,
        'early_stopping_patience': None
    },{
        'name': 'ADAM Optimizer',
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 500,
        'batch_size': 64,
        'optimizer': 'adam',
        'lr_schedule': None,
        'early_stopping_patience': None
    },{
        'name': 'L2 Regularization',
        'learning_rate': 0.001,
        'l2_lambda': 0.001,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 1000,
        'batch_size': 64,
        'optimizer': 'sgd',
        'lr_schedule': None,
        'early_stopping_patience': None
    },{
        'name': 'Early Stopping',
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': False,
        'epochs': 4000,
        'batch_size': 64,
        'optimizer': 'sgd',
        'lr_schedule': None,
        'early_stopping_patience': 10
    },{
        'name': 'Dropout',
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.2,
        'use_batch_norm': False,
        'epochs': 2000,
        'batch_size': 64,
        'optimizer': 'sgd',
        'lr_schedule': None,
        'early_stopping_patience': None
    },{
```

```
        'name': 'Batch Normalization',
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.0,
        'use_batch_norm': True,
        'epochs': 750,
        'batch_size': 64,
        'optimizer': 'sgd',
        'lr_schedule': None,
        'early_stopping_patience': None
    },{
        'name': 'Combined Improvements',
        'learning_rate': 0.001,
        'l2_lambda': 0.0005,
        'dropout_rate': 0.2,
        'use_batch_norm': True,
        'epochs': 4000,
        'batch_size': 64,
        'optimizer': 'adam',
        'lr_schedule': 'exponential',
        'early_stopping_patience': 15
    }
]
```

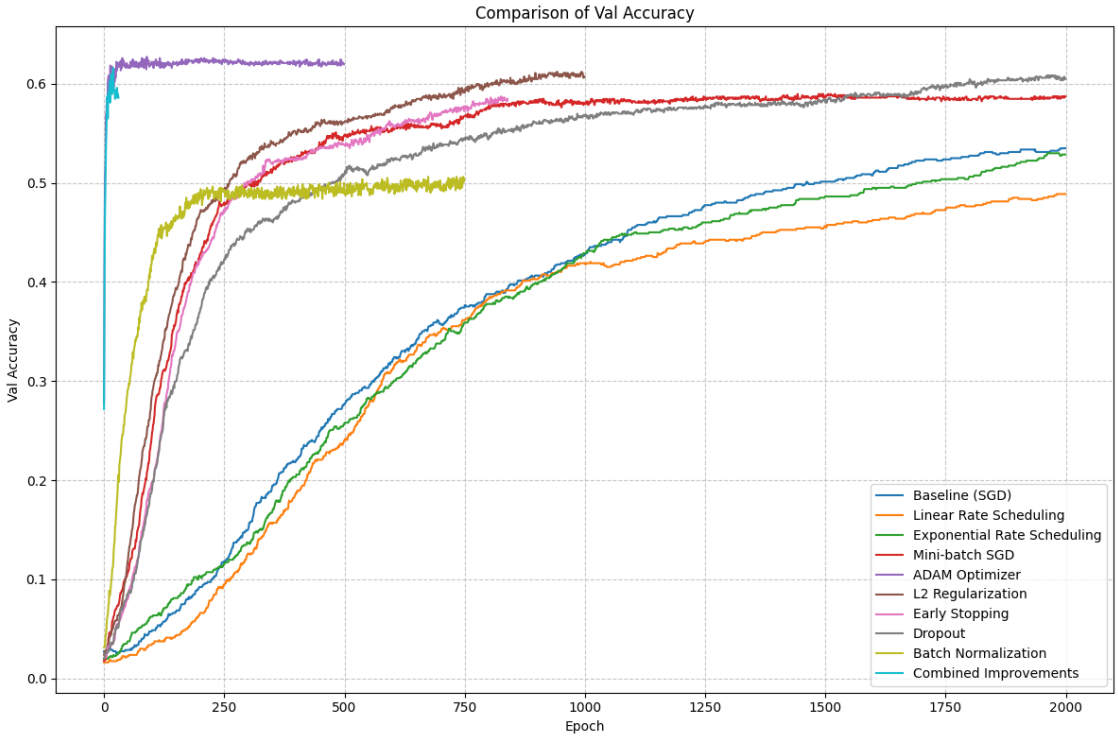
[15]:
```
results = run_experiments(X_train, y_train, X_val, y_val,␣
 ↪ImprovedNeuralNetwork, layer_sizes, experiments)
```

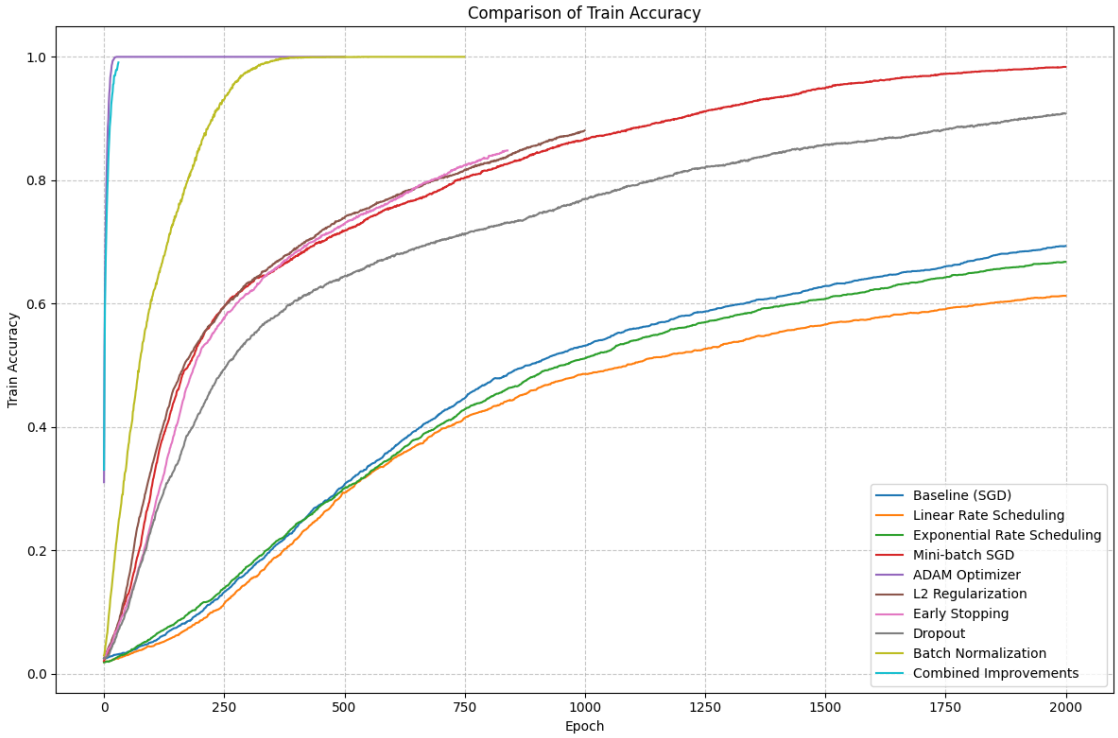
```
Running experiment: Baseline (SGD)
Epoch: [==================================================>] 100% - train_loss:
1.2644 - train_acc: 0.6935 - val_loss: 1.8312 - val_acc: 0.5349 - lr: 0.0100

Results for Baseline (SGD):
Training time: 202.27 seconds
Final train loss: 1.2644, train accuracy: 0.6935
Final val loss: 1.8312, val accuracy: 0.5349

Running experiment: Linear Rate Scheduling
Epoch: [==================================================>] 100% - train_loss:
1.6270 - train_acc: 0.6128 - val_loss: 2.0439 - val_acc: 0.4888 - lr: 0.0050

Results for Linear Rate Scheduling:
Training time: 210.50 seconds
Final train loss: 1.6270, train accuracy: 0.6128
Final val loss: 2.0439, val accuracy: 0.4888

Running experiment: Exponential Rate Scheduling
```

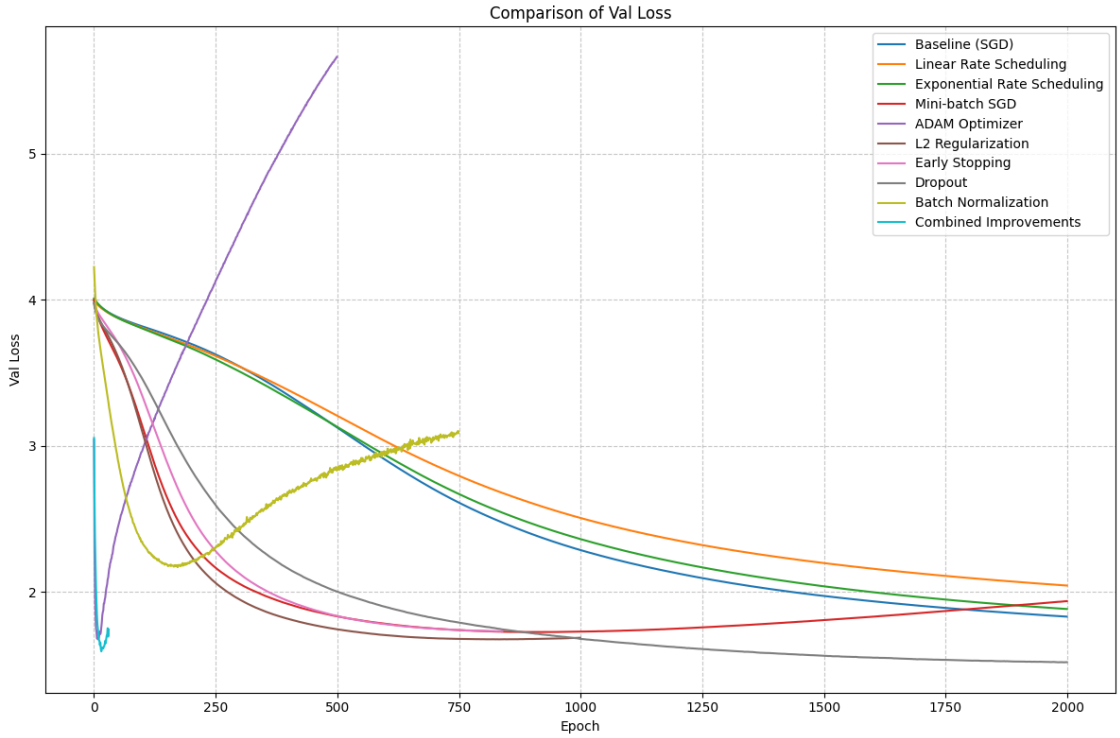
```
Epoch: [==================================================>] 100% - train_loss:
1.3775 - train_acc: 0.6678 - val_loss: 1.8833 - val_acc: 0.5287 - lr: 0.0090

Results for Exponential Rate Scheduling:
Training time: 213.82 seconds
Final train loss: 1.3775, train accuracy: 0.6678
Final val loss: 1.8833, val accuracy: 0.5287

Running experiment: Mini-batch SGD
Epoch: [==================================================>] 100% - train_loss:
0.1835 - train_acc: 0.9837 - val_loss: 1.9371 - val_acc: 0.5873 - lr: 0.0010

Results for Mini-batch SGD:
Training time: 346.95 seconds
Final train loss: 0.1835, train accuracy: 0.9837
Final val loss: 1.9371, val accuracy: 0.5873

Running experiment: ADAM Optimizer
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 5.6615 - val_acc: 0.6197 - lr: 0.0010

Results for ADAM Optimizer:
Training time: 130.97 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 5.6615, val accuracy: 0.6197

Running experiment: L2 Regularization
Epoch: [==================================================>] 100% - train_loss:
0.5758 - train_acc: 0.8805 - val_loss: 1.6866 - val_acc: 0.6072 - lr: 0.0010

Results for L2 Regularization:
Training time: 187.04 seconds
Final train loss: 0.5758, train accuracy: 0.8805
Final val loss: 1.6866, val accuracy: 0.6072

Running experiment: Early Stopping
Epoch: [=========>                                          ] 21% - train_loss:
0.7000 - train_acc: 0.8483 - val_loss: 1.7314 - val_acc: 0.5848 - lr: 0.0010
Early stopping triggered at epoch 840


Results for Early Stopping:
Training time: 147.27 seconds
Final train loss: 0.7000, train accuracy: 0.8483
Final val loss: 1.7314, val accuracy: 0.5848

Running experiment: Dropout
Epoch: [==================================================>] 100% - train_loss:
```

```
0.4686 - train_acc: 0.9080 - val_loss: 1.5195 - val_acc: 0.6047 - lr: 0.0010

Results for Dropout:
Training time: 411.22 seconds
Final train loss: 0.4686, train accuracy: 0.9080
Final val loss: 1.5195, val accuracy: 0.6047

Running experiment: Batch Normalization
Epoch: [==================================================>] 100% - train_loss:
0.0084 - train_acc: 1.0000 - val_loss: 3.0874 - val_acc: 0.5037 - lr: 0.0010

Results for Batch Normalization:
Training time: 181.09 seconds
Final train loss: 0.0084, train accuracy: 1.0000
Final val loss: 3.0874, val accuracy: 0.5037

Running experiment: Combined Improvements
Epoch: [>                                                  ] 0% - train_loss:
0.1098 - train_acc: 0.9909 - val_loss: 1.7435 - val_acc: 0.5860 - lr: 0.0010
Early stopping triggered at epoch 31


Results for Combined Improvements:
Training time: 11.62 seconds
Final train loss: 0.1098, train accuracy: 0.9909
Final val loss: 1.7435, val accuracy: 0.5860
```

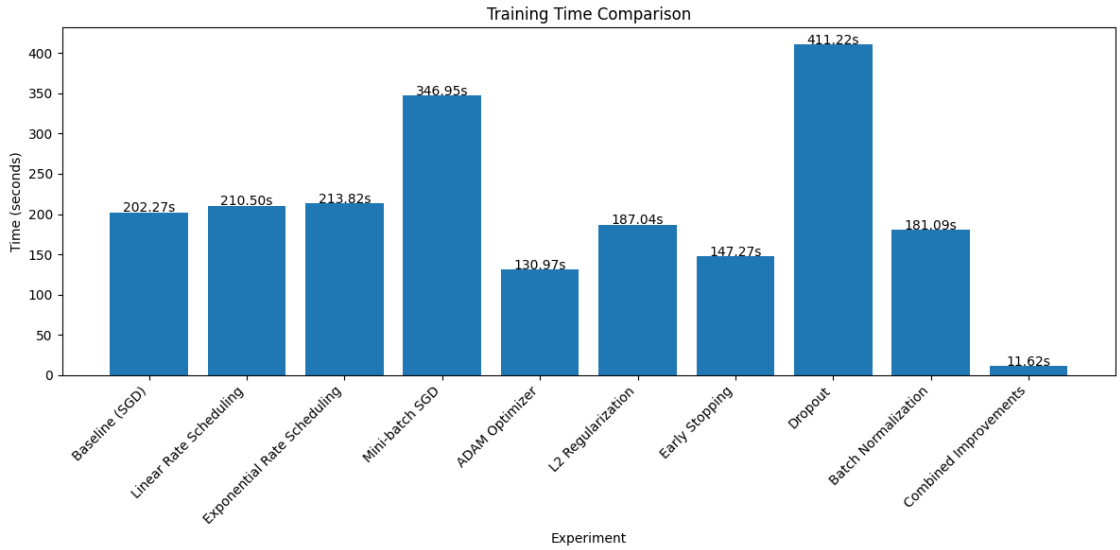
[16]:
```python
plot_experiment_results(results, metric='val_accuracy')
plot_experiment_results(results, metric='train_accuracy')
plot_experiment_results(results, metric='val_loss')
```

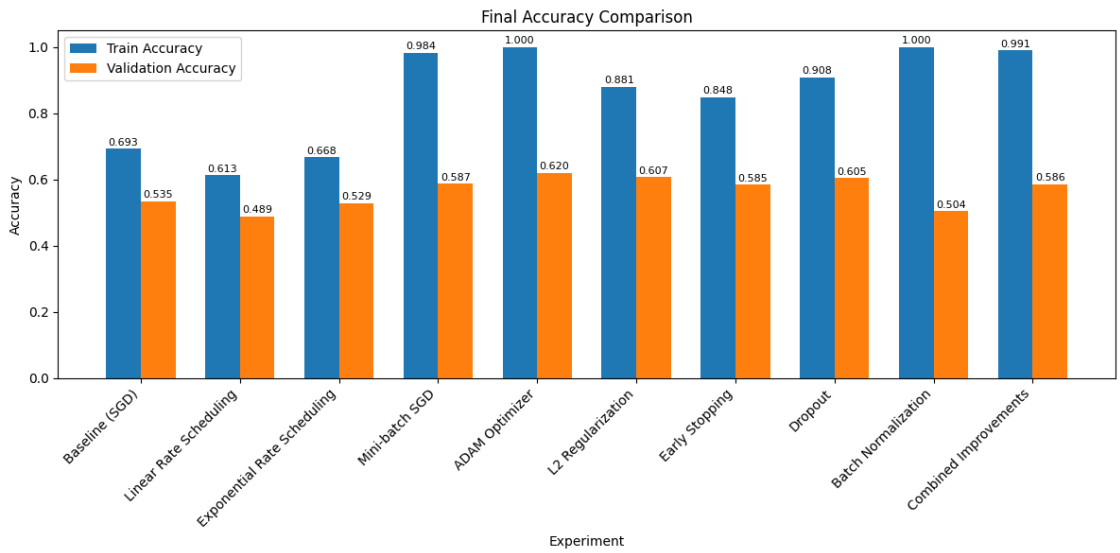

Comparison of Val Accuracy
Val Accuracy
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
250
500
750
1000
1250
1500
1750
2000
Baseline (SGD)
Linear Rate Scheduling
Exponential Rate Scheduling
Mini-batch SGD
ADAM Optimizer
L2 Regularization
Early Stopping
Dropout
Batch Normalization
Combined Improvements


Comparison of Train Accuracy
Train Accuracy
Epoch
0.0
0.2
0.4
0.6
0.8
1.0
0
250
500
750
1000
1250
1500
1750
2000
Baseline (SGD)
Linear Rate Scheduling
Exponential Rate Scheduling
Mini-batch SGD
ADAM Optimizer
L2 Regularization
Early Stopping
Dropout
Batch Normalization
Combined Improvements



[17]:
```
compare_training_times(results)
compare_final_metrics(results)
```





[18]:
```
sorted_results = sorted(results, key=lambda x: x['final_val_accuracy'],␣
↪reverse=True)

best_result = sorted_results[0]
print(f"\nBest model: {best_result['name']}")
print(f"Validation accuracy: {best_result['final_val_accuracy']:.4f}")

second_best_result = sorted_results[1]
print(f"\nSecond best model: {second_best_result['name']}")
print(f"Validation accuracy: {second_best_result['final_val_accuracy']:.4f}")
```

```
Best model: ADAM Optimizer
Validation accuracy: 0.6197

Second best model: L2 Regularization
Validation accuracy: 0.6072
```

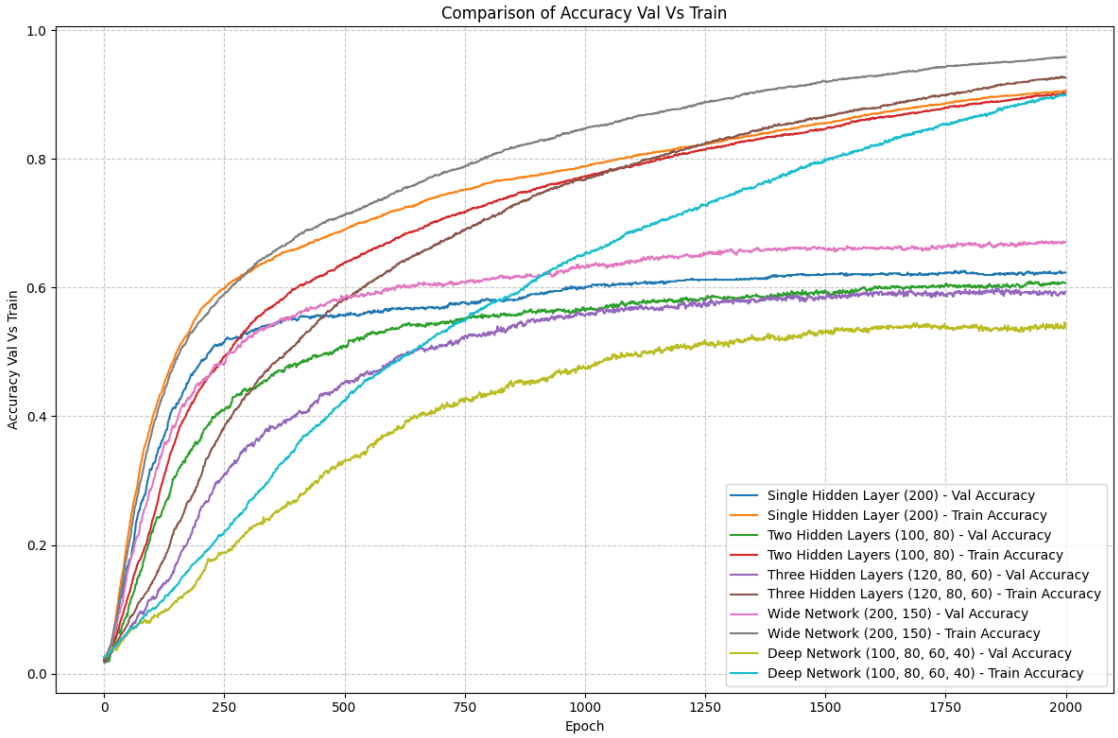
[22]:
```
print("\nExploring different architectures with the best configuration...")
architectures = [
    {
        'name': 'Single Hidden Layer (200)',
        'layer_sizes': [input_size, 200, output_size]
    },{
        'name': 'Two Hidden Layers (100, 80)',
        'layer_sizes': [input_size, 100, 80, output_size]
    },{
        'name': 'Three Hidden Layers (120, 80, 60)',
        'layer_sizes': [input_size, 120, 80, 60, output_size]
```

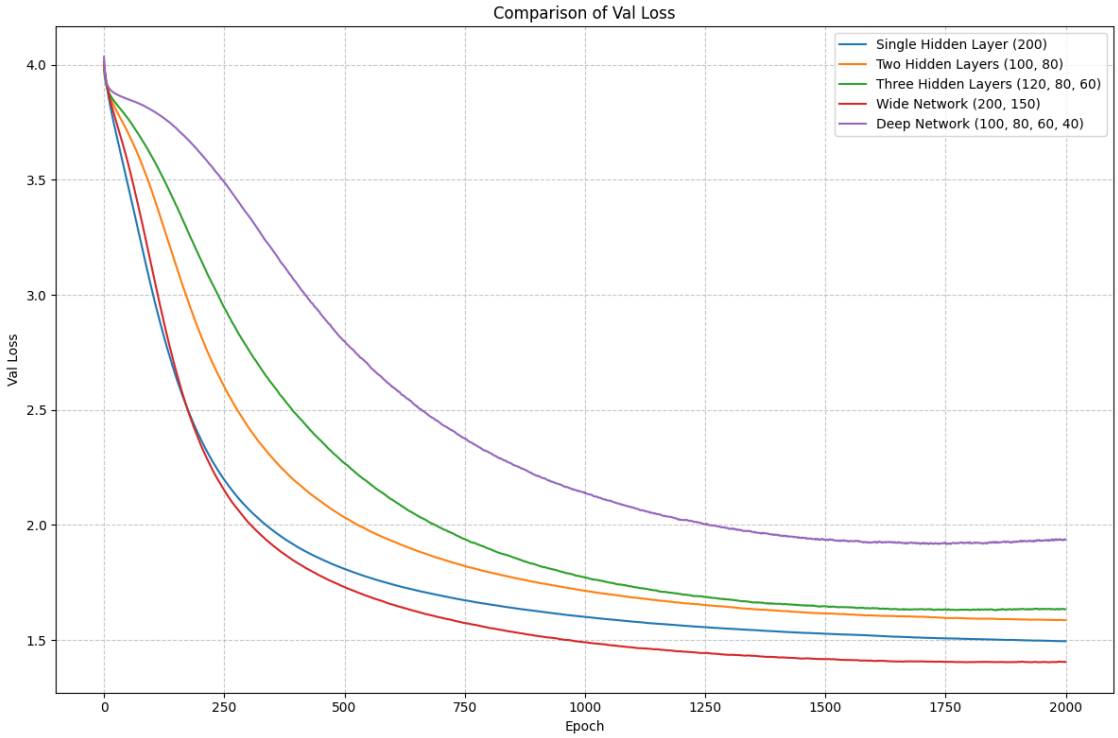
```
    },{
        'name': 'Wide Network (200, 150)',
        'layer_sizes': [input_size, 200, 150, output_size]
    },{
        'name': 'Deep Network (100, 80, 60, 40)',
        'layer_sizes': [input_size, 100, 80, 60, 40, output_size]
    }
]

architecture_results = run_architecture_experiments(
    architectures=architectures,
    cofiguration=best_result,
    ImprovedNeuralNetwork=ImprovedNeuralNetwork,
    X_train=X_train,
    y_train=y_train,
    X_val=X_val,
    y_val=y_val
)
```

```
Exploring different architectures with the best configuration…

==================================================
Testing architecture: Single Hidden Layer (200)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 4.2547 - val_acc: 0.6309 - lr: 0.0010

Results for Single Hidden Layer (200):
Training time: 171.35 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 4.2547, val accuracy: 0.6309

==================================================
Testing architecture: Two Hidden Layers (100, 80)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 5.5579 - val_acc: 0.6384 - lr: 0.0010

Results for Two Hidden Layers (100, 80):
Training time: 129.44 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 5.5579, val accuracy: 0.6384

==================================================
Testing architecture: Three Hidden Layers (120, 80, 60)
==================================================
```

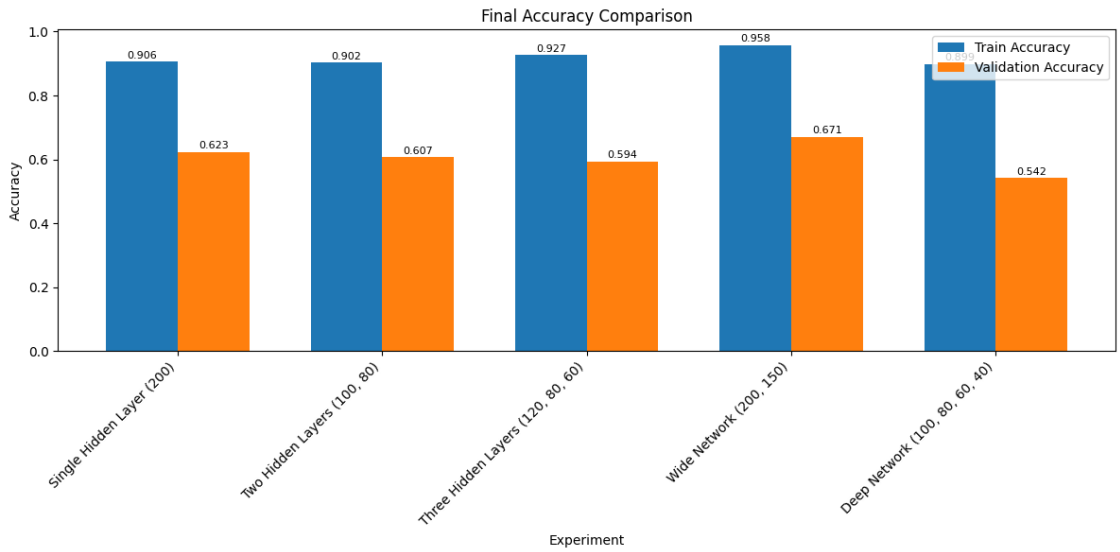
```
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 5.9901 - val_acc: 0.6160 - lr: 0.0010

Results for Three Hidden Layers (120, 80, 60):
Training time: 160.44 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 5.9901, val accuracy: 0.6160

==================================================
Testing architecture: Wide Network (200, 150)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 4.6014 - val_acc: 0.6683 - lr: 0.0010

Results for Wide Network (200, 150):
Training time: 222.59 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 4.6014, val accuracy: 0.6683

==================================================
Testing architecture: Deep Network (100, 80, 60, 40)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 7.1449 - val_acc: 0.5898 - lr: 0.0010

Results for Deep Network (100, 80, 60, 40):
Training time: 151.61 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 7.1449, val accuracy: 0.5898
```

[ ]:
```
plot_experiment_results(architecture_results, metric='accuracy_val_vs_train')
plot_experiment_results(architecture_results, metric='val_loss')
```


Comparison of Accuracy Val Vs Train
Accuracy Val Vs Train
Epoch
Single Hidden Layer (200) - Val Accuracy
Single Hidden Layer (200) - Train Accuracy
Two Hidden Layers (100, 80) - Val Accuracy
Two Hidden Layers (100, 80) - Train Accuracy
Three Hidden Layers (120, 80, 60) - Val Accuracy
Three Hidden Layers (120, 80, 60) - Train Accuracy
Wide Network (200, 150) - Val Accuracy
Wide Network (200, 150) - Train Accuracy
Deep Network (100, 80, 60, 40) - Val Accuracy
Deep Network (100, 80, 60, 40) - Train Accuracy


Comparison of Val Loss
Val Loss
Epoch
Single Hidden Layer (200)
Two Hidden Layers (100, 80)
Three Hidden Layers (120, 80, 60)
Wide Network (200, 150)
Deep Network (100, 80, 60, 40)

[ ]:
```
compare_final_metrics(architecture_results)

best_arch_result = max(architecture_results, key=lambda x:␣
↪x['final_val_accuracy'])
print(f"\nBest architecture: {best_arch_result['name']}")
print(f"Validation accuracy: {best_arch_result['final_val_accuracy']:.4f}")
```




```
Best architecture: Wide Network (200, 150)
Validation accuracy: 0.6708
```

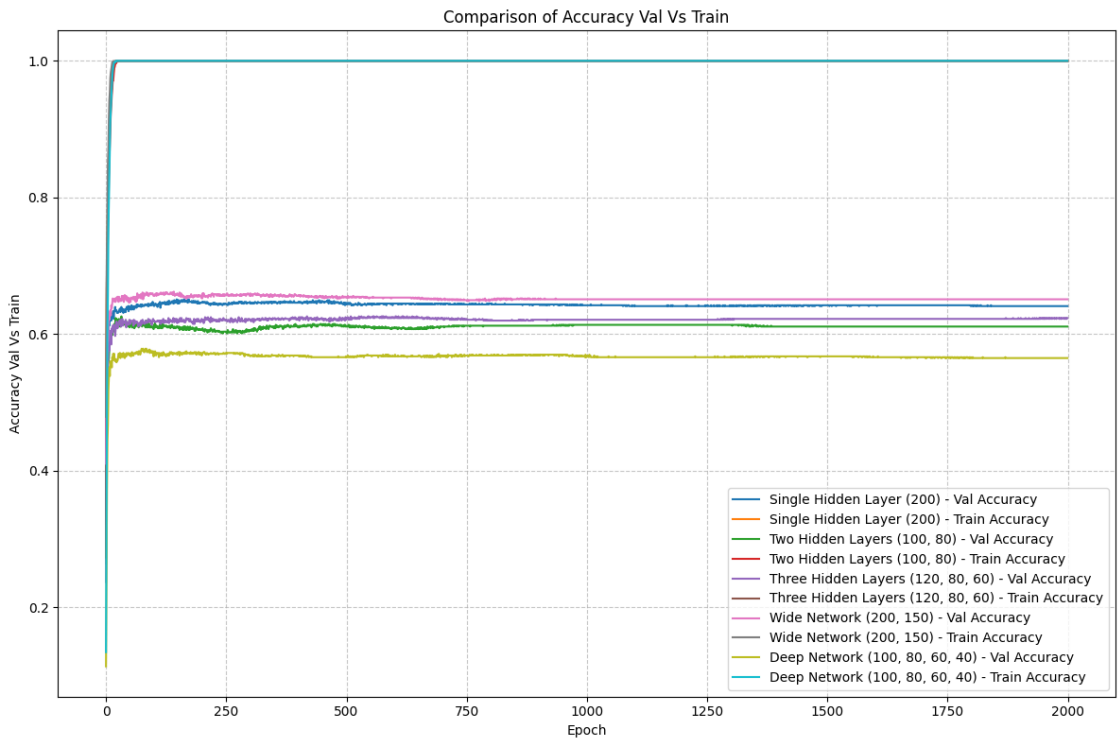
[ ]:
```
print("\nExploring different architectures with the second best configuration...
↪")

architecture_results_2 = run_architecture_experiments(
    architectures=architectures,
    cofiguration=second_best_result,
    ImprovedNeuralNetwork=ImprovedNeuralNetwork,
    X_train=X_train,
    y_train=y_train,
    X_val=X_val,
    y_val=y_val
)
```

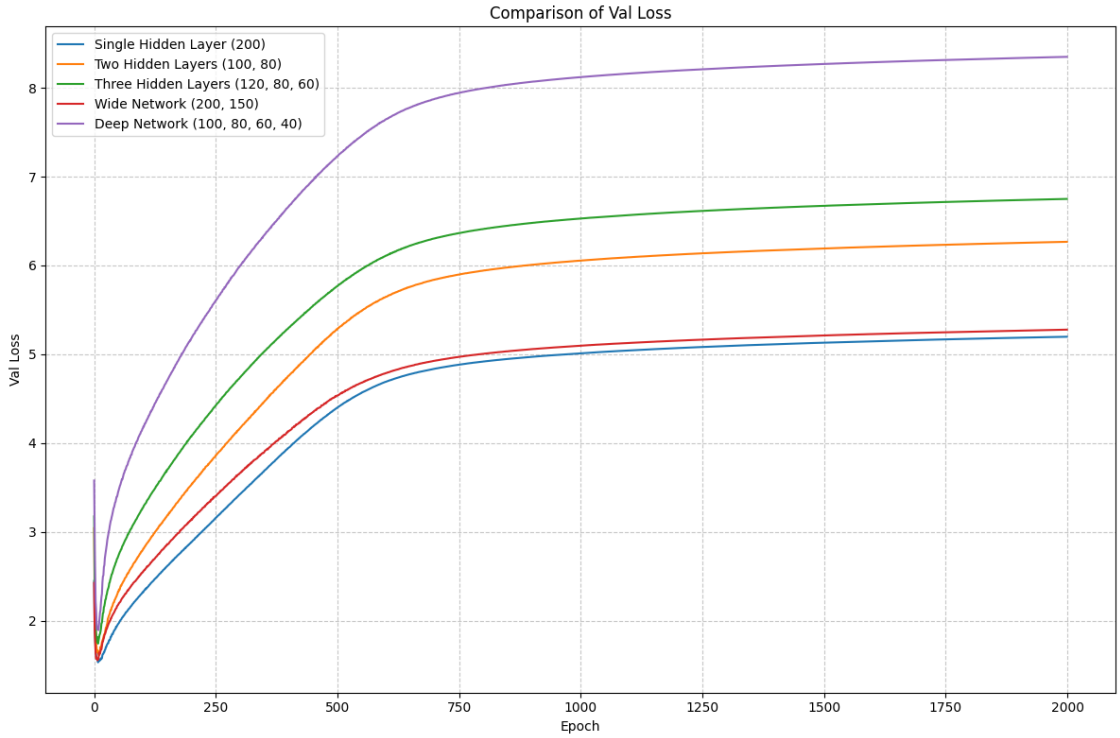
```
Exploring different architectures with the second best configuration…

==================================================
Testing architecture: Single Hidden Layer (200)
```

```
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 5.1974 - val_acc: 0.6409 - lr: 0.0010

Results for Single Hidden Layer (200):
Training time: 617.74 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 5.1974, val accuracy: 0.6409

==================================================
Testing architecture: Two Hidden Layers (100, 80)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 6.2664 - val_acc: 0.6110 - lr: 0.0010

Results for Two Hidden Layers (100, 80):
Training time: 446.93 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 6.2664, val accuracy: 0.6110

==================================================
Testing architecture: Three Hidden Layers (120, 80, 60)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 6.7492 - val_acc: 0.6234 - lr: 0.0010

Results for Three Hidden Layers (120, 80, 60):
Training time: 536.03 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 6.7492, val accuracy: 0.6234

==================================================
Testing architecture: Wide Network (200, 150)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 5.2774 - val_acc: 0.6509 - lr: 0.0010

Results for Wide Network (200, 150):
Training time: 762.12 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 5.2774, val accuracy: 0.6509

==================================================
Testing architecture: Deep Network (100, 80, 60, 40)
==================================================
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 8.3487 - val_acc: 0.5648 - lr: 0.0010
```

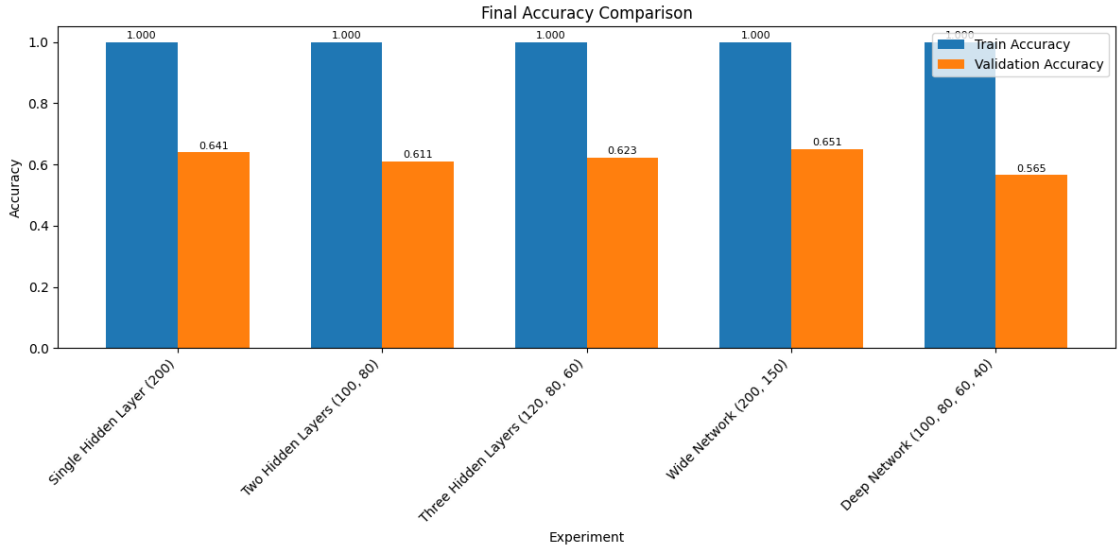
```
Results for Deep Network (100, 80, 60, 40):
Training time: 518.88 seconds
Final train loss: 0.0000, train accuracy: 1.0000
Final val loss: 8.3487, val accuracy: 0.5648
```

[ ]:
```
plot_experiment_results(architecture_results_2, metric='accuracy_val_vs_train')
plot_experiment_results(architecture_results_2, metric='val_loss')
```





[ ]:
```
compare_final_metrics(architecture_results_2)

best_arch_result_2 = max(architecture_results_2, key=lambda x:␣
 ↪x['final_val_accuracy'])
print(f"\nBest architecture: {best_arch_result_2['name']}")
print(f"Validation accuracy: {best_arch_result_2['final_val_accuracy']:.4f}")
```



```
Best architecture: Wide Network (200, 150)
Validation accuracy: 0.6509
```

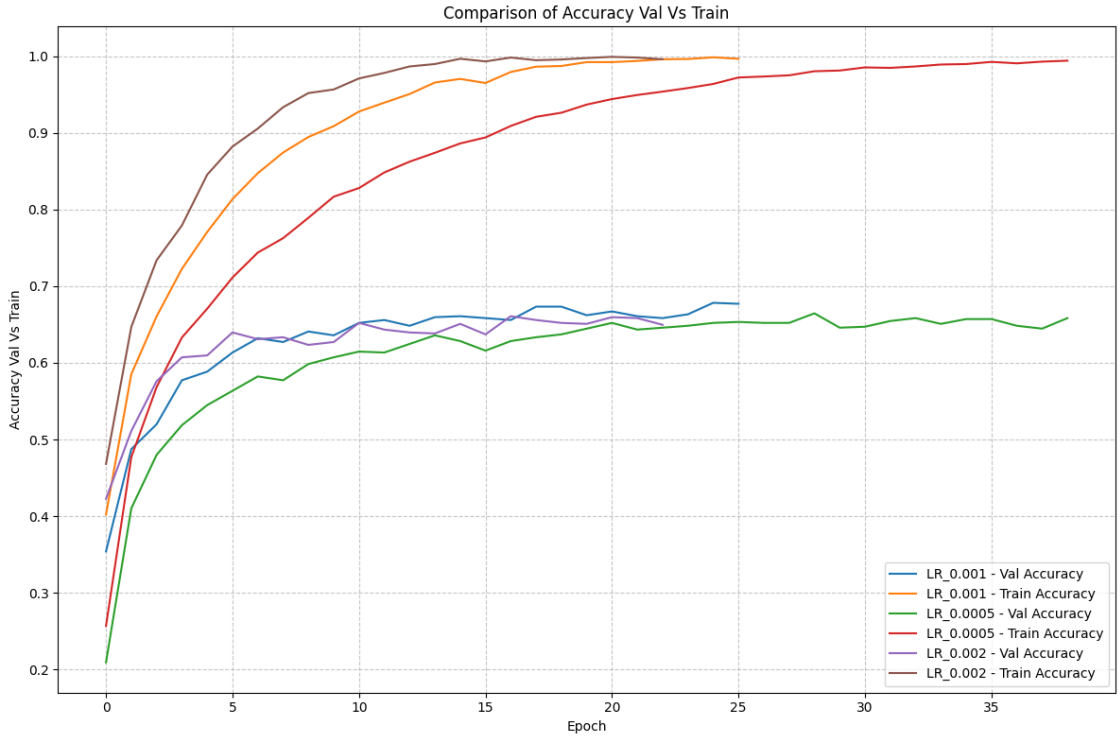
[ ]:
```
learning_rates_to_test = [0.001, 0.0005, 0.002]

lr_tuning_experiments = []

config = {
        'learning_rate': 0.001,
        'l2_lambda': 0.0,
        'dropout_rate': 0.2,
        'use_batch_norm': False,
        'epochs': 4000,
        'batch_size': 64,
        'optimizer': 'adam',
        'lr_schedule': 'exponential',
        'early_stopping_patience': 15
    }

for lr_value in learning_rates_to_test:
    lr_tuning_experiments.append(config.copy())
    lr_tuning_experiments[-1]['learning_rate'] = lr_value
    lr_tuning_experiments[-1]['name'] = f"LR_{lr_value}"

lr_tuning_results = run_experiments(X_train, y_train, X_val, y_val,␣
 ↪ImprovedNeuralNetwork, best_arch_result['layer_sizes'],␣
 ↪lr_tuning_experiments)
```

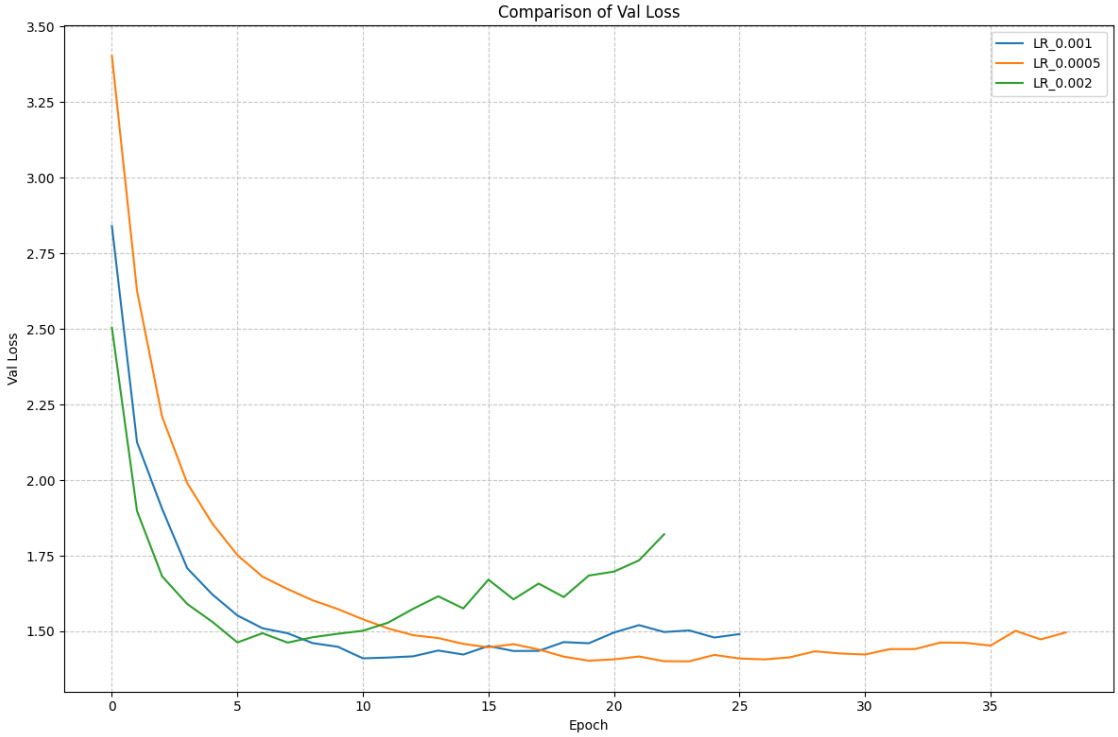
```
Running experiment: LR_0.001
Epoch: [>                                                  ] 0% - train_loss:
0.0436 - train_acc: 0.9966 - val_loss: 1.4910 - val_acc: 0.6771 - lr: 0.0010
Early stopping triggered at epoch 26

Best result for LR_0.001: val_loss = 1.4107, val_accuracy = 0.6521

Running experiment: LR_0.0005
Epoch: [>                                                  ] 0% - train_loss:
0.0751 - train_acc: 0.9941 - val_loss: 1.4964 - val_acc: 0.6584 - lr: 0.0005
Early stopping triggered at epoch 39

Best result for LR_0.0005: val_loss = 1.4008, val_accuracy = 0.6484

Running experiment: LR_0.002
Epoch: [>                                                  ] 0% - train_loss:
```

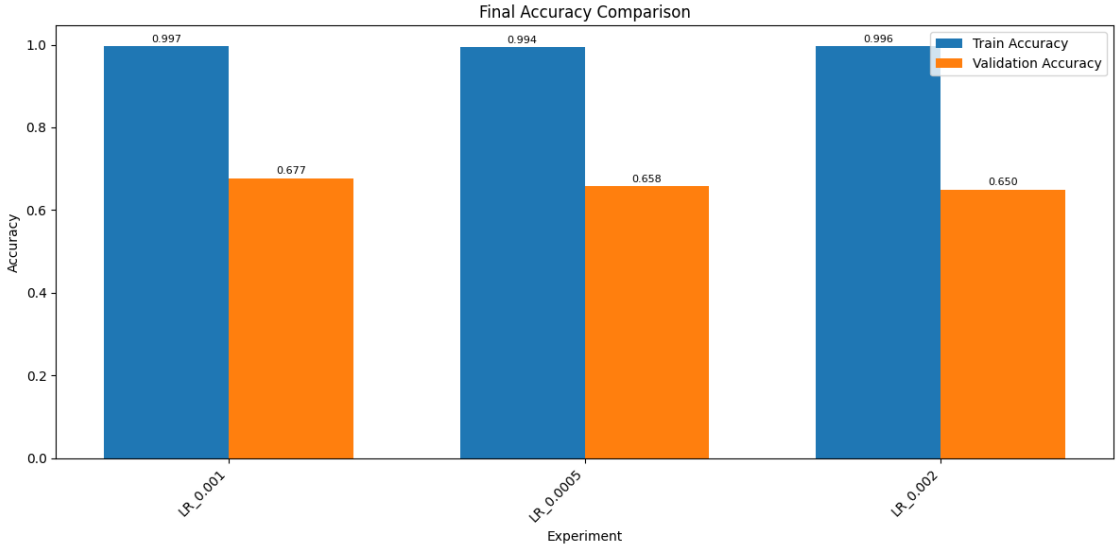
```
0.0228 - train_acc: 0.9959 - val_loss: 1.8211 - val_acc: 0.6496 - lr: 0.0020
Early stopping triggered at epoch 23

Best result for LR_0.002: val_loss = 1.4627, val_accuracy = 0.6334
```

[ ]:
```
print("\nGraficando resultados del ajuste de LR (sin dropout / l2)...")
plot_experiment_results(lr_tuning_results, metric='accuracy_val_vs_train')
plot_experiment_results(lr_tuning_results, metric='val_loss')
compare_final_metrics(lr_tuning_results)

best_lr_tuned_model = max(lr_tuning_results, key=lambda x: x.
 ↪get('final_val_accuracy', -float('inf')))
print(f"\n--- Mejor Modelo Después del Ajuste de Tasa de Aprendizaje (sin␣
 ↪dropout/l2) ---")
print(f"Nombre: {best_lr_tuned_model.get('name', 'N/A')}")
print(f"Precisión de Validación: {best_lr_tuned_model.get('final_val_accuracy',␣
 ↪'N/A'):.4f}")
print(f"Tasa de Aprendizaje: {best_lr_tuned_model.get('learning_rate', 'N/A')}")
print(f"Configuración Completa:")
for key, value in best_lr_tuned_model.items():
    if key not in ['history']:
            print(f"  {key}: {value}")
```

```
Graficando resultados del ajuste de LR (sin dropout / l2)…
```








```
--- Mejor Modelo Después del Ajuste de Tasa de Aprendizaje (sin dropout/l2) ---
Nombre: LR_0.001
```

```
Precisión de Validación: 0.6771
Tasa de Aprendizaje: 0.001
Configuración Completa:
  learning_rate: 0.001
  l2_lambda: 0.0
  dropout_rate: 0.2
  use_batch_norm: False
  epochs: 4000
  batch_size: 64
  optimizer: adam
  lr_schedule: exponential
  early_stopping_patience: 15
  name: LR_0.001
  model: <src.improved_neural_network.ImprovedNeuralNetwork object at
0x70d0fd7d56a0>
  final_train_loss: 0.043599454467499875
  final_train_accuracy: 0.9965592743196747
  final_val_loss: 1.4910018758302466
  final_val_accuracy: 0.6770573566084788
  training_time: 11.047519445419312
```

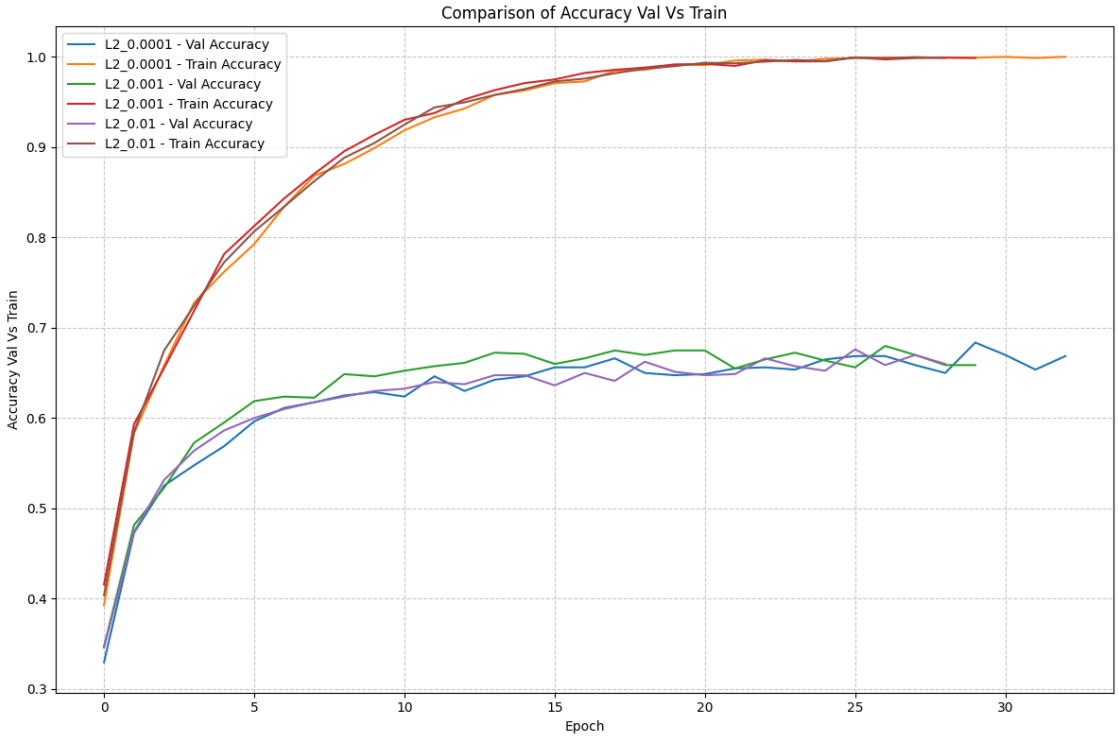
[ ]:
```python
l2_values_to_test = [0.0001, 0.001, 0.01]

l2_tuning_experiments = []

best_lr = best_lr_tuned_model['learning_rate']

config['learning_rate']= best_lr

for l2_value in l2_values_to_test:
    l2_tuning_experiments.append(config.copy())
    l2_tuning_experiments[-1]['l2_lambda'] = l2_value
    l2_tuning_experiments[-1]['name'] = f"L2_{l2_value}"

l2_tuning_results = run_experiments(
    X_train, y_train, X_val, y_val,
    ImprovedNeuralNetwork,
    best_arch_result['layer_sizes'],
    l2_tuning_experiments
)
```

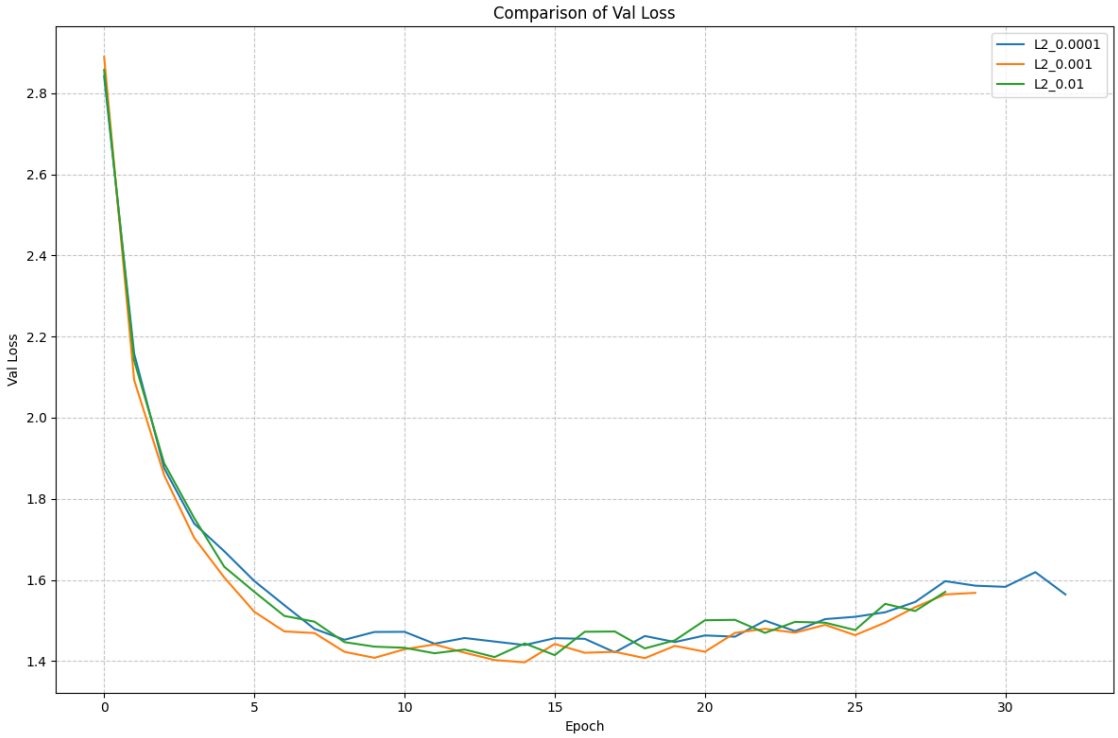
```
Running experiment: L2_0.0001
Epoch: [>                                                  ] 0% - train_loss:
0.0219 - train_acc: 0.9997 - val_loss: 1.5642 - val_acc: 0.6683 - lr: 0.0010
Early stopping triggered at epoch 33

Best result for L2_0.0001: val_loss = 1.4216, val_accuracy = 0.6658
```

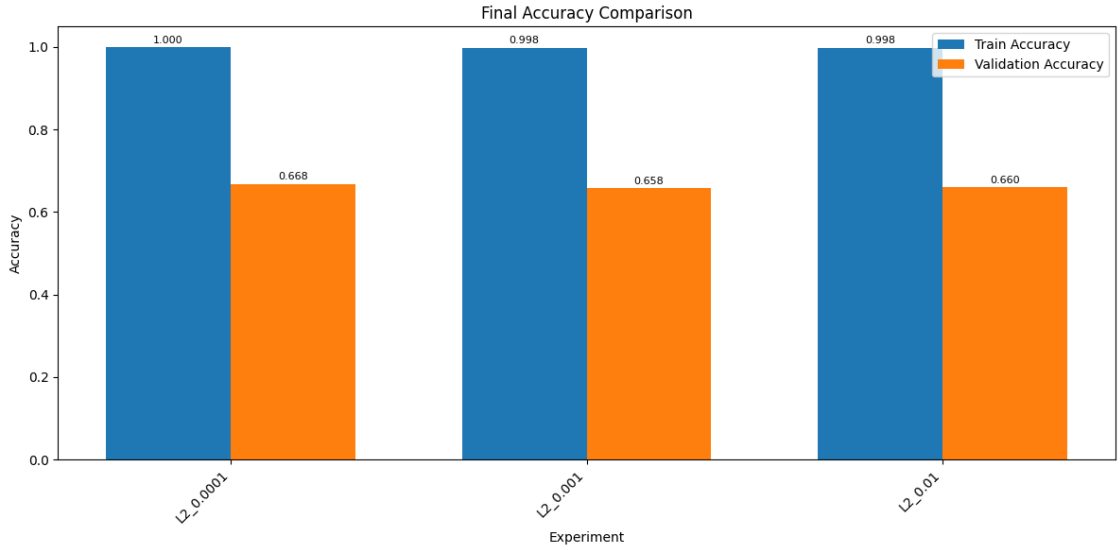
```
Running experiment: L2_0.001
Epoch: [>                                                  ] 0% - train_loss:
0.0267 - train_acc: 0.9984 - val_loss: 1.5682 - val_acc: 0.6584 - lr: 0.0010
Early stopping triggered at epoch 30

Best result for L2_0.001: val_loss = 1.3968, val_accuracy = 0.6708

Running experiment: L2_0.01
Epoch: [>                                                  ] 0% - train_loss:
0.0359 - train_acc: 0.9984 - val_loss: 1.5707 - val_acc: 0.6596 - lr: 0.0010
Early stopping triggered at epoch 29

Best result for L2_0.01: val_loss = 1.4099, val_accuracy = 0.6471
```

[ ]:
```
print("\nGraficando resultados del ajuste de L2...")
plot_experiment_results(l2_tuning_results, metric='accuracy_val_vs_train')
plot_experiment_results(l2_tuning_results, metric='val_loss')
compare_final_metrics(l2_tuning_results)

best_l2_tuned_model = max(l2_tuning_results, key=lambda x: x.
 ↪get('final_val_accuracy', -float('inf')))
print(f"\n--- Mejor Modelo Después del Ajuste de L2 ---")
print(f"Nombre: {best_l2_tuned_model.get('name', 'N/A')}")
print(f"Precisión de Validación: {best_l2_tuned_model.get('final_val_accuracy',␣
 ↪'N/A'):.4f}")
print(f"L2 Lambda: {best_l2_tuned_model.get('l2_lambda', 'N/A')}")
print(f"Configuración Completa:")
for key, value in best_l2_tuned_model.items():
    if key not in ['history']:
        print(f"  {key}: {value}")
```

```
Graficando resultados del ajuste de L2…
```


Comparison of Accuracy Val Vs Train
L2_0.0001 - Val Accuracy
L2_0.0001 - Train Accuracy
L2_0.001 - Val Accuracy
L2_0.001 - Train Accuracy
L2_0.01 - Val Accuracy
L2_0.01 - Train Accuracy
Accuracy Val Vs Train
Epoch


Comparison of Val Loss
L2_0.0001
L2_0.001
L2_0.01
Val Loss
Epoch




```
--- Mejor Modelo Después del Ajuste de L2 ---
Nombre: L2_0.0001
Precisión de Validación: 0.6683
L2 Lambda: 0.0001
Configuración Completa:
  learning_rate: 0.001
  l2_lambda: 0.0001
  dropout_rate: 0.2
  use_batch_norm: False
  epochs: 4000
  batch_size: 64
  optimizer: adam
  lr_schedule: exponential
  early_stopping_patience: 15
  name: L2_0.0001
  model: <src.improved_neural_network.ImprovedNeuralNetwork object at
0x70d0fd7d58b0>
  final_train_loss: 0.021887515367176282
  final_train_accuracy: 0.999687206756334
  final_val_loss: 1.564217158163102
  final_val_accuracy: 0.6683291770573566
  training_time: 15.3146333694458
```

[ ]:
```
dropout_values_to_test = [0.1, 0.3, 0.5]

dropout_tuning_experiments = []

best_l2 = best_l2_tuned_model['l2_lambda']
```

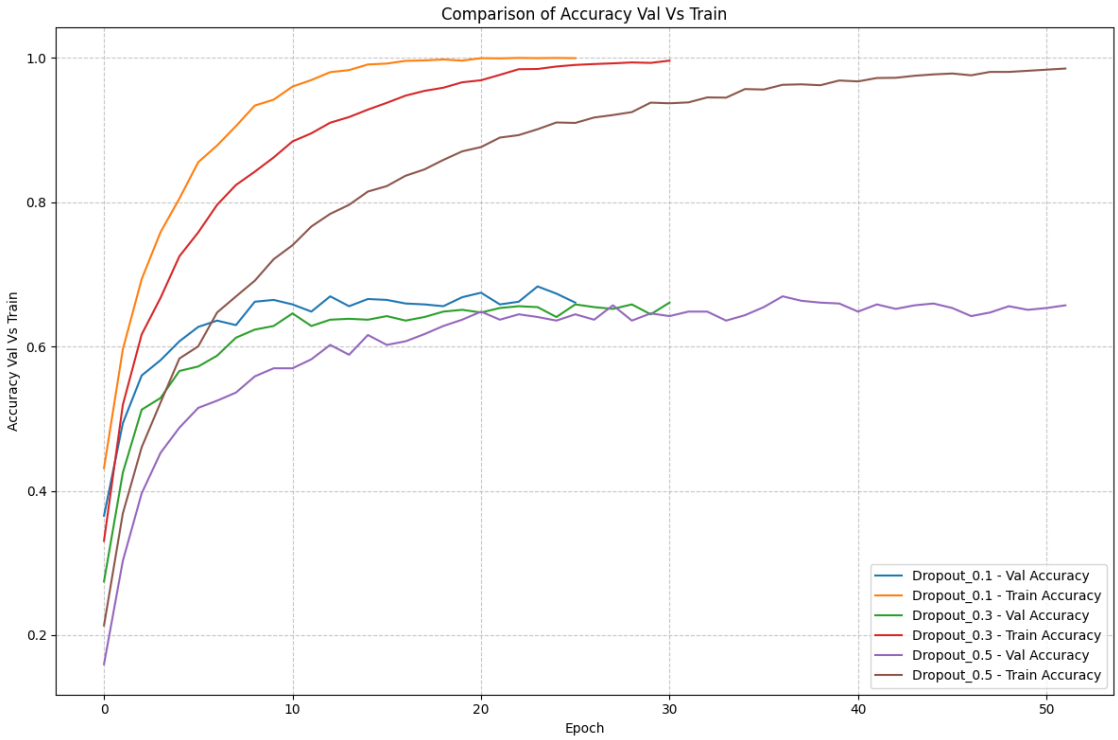
```
config['l2_lambda'] = best_l2

for dropout_value in dropout_values_to_test:
    dropout_tuning_experiments.append(config.copy())
    dropout_tuning_experiments[-1]['dropout_rate'] = dropout_value
    dropout_tuning_experiments[-1]['name'] = f"Dropout_{dropout_value}"

dropout_tuning_results = run_experiments(
    X_train, y_train, X_val, y_val,
    ImprovedNeuralNetwork,
    best_arch_result['layer_sizes'],
    dropout_tuning_experiments
)
```

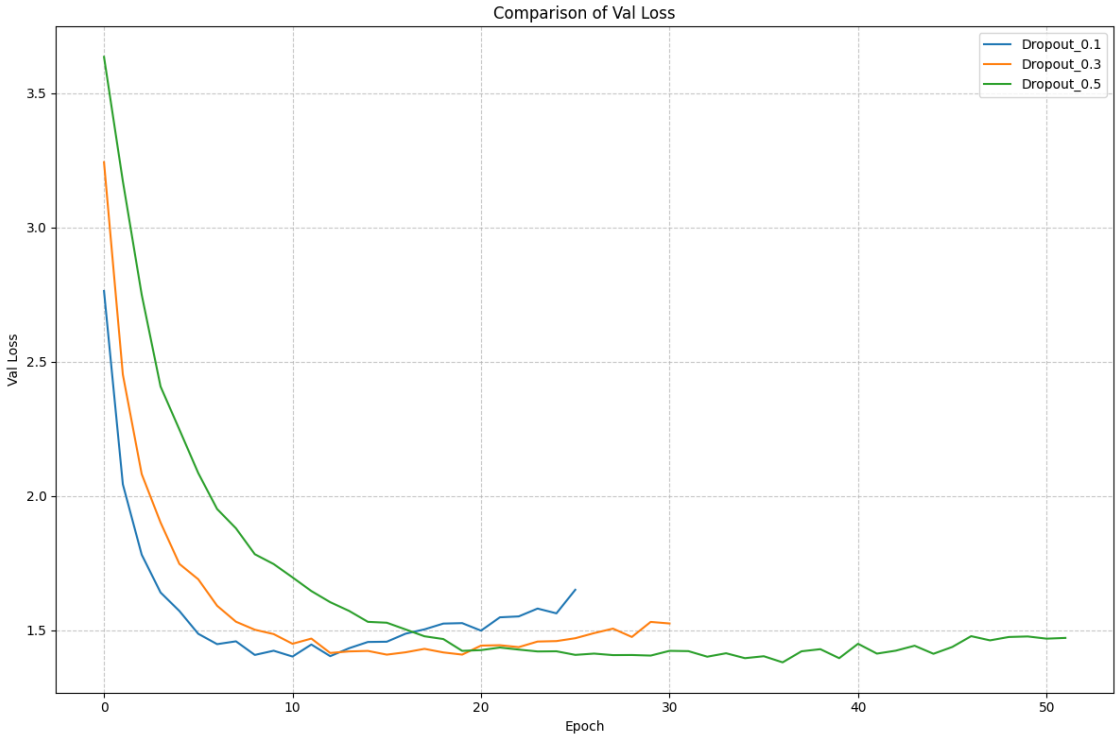
```
Running experiment: Dropout_0.1
Epoch: [>                                                  ] 0% - train_loss:
0.0202 - train_acc: 0.9997 - val_loss: 1.6506 - val_acc: 0.6608 - lr: 0.0010
Early stopping triggered at epoch 26

Best result for Dropout_0.1: val_loss = 1.4020, val_accuracy = 0.6584

Running experiment: Dropout_0.3
Epoch: [>                                                  ] 0% - train_loss:
0.0572 - train_acc: 0.9962 - val_loss: 1.5252 - val_acc: 0.6608 - lr: 0.0010
Early stopping triggered at epoch 31

Best result for Dropout_0.3: val_loss = 1.4092, val_accuracy = 0.6421

Running experiment: Dropout_0.5
Epoch: [>                                                  ] 1% - train_loss:
0.1212 - train_acc: 0.9853 - val_loss: 1.4714 - val_acc: 0.6571 - lr: 0.0010
Early stopping triggered at epoch 52

Best result for Dropout_0.5: val_loss = 1.3804, val_accuracy = 0.6696
```

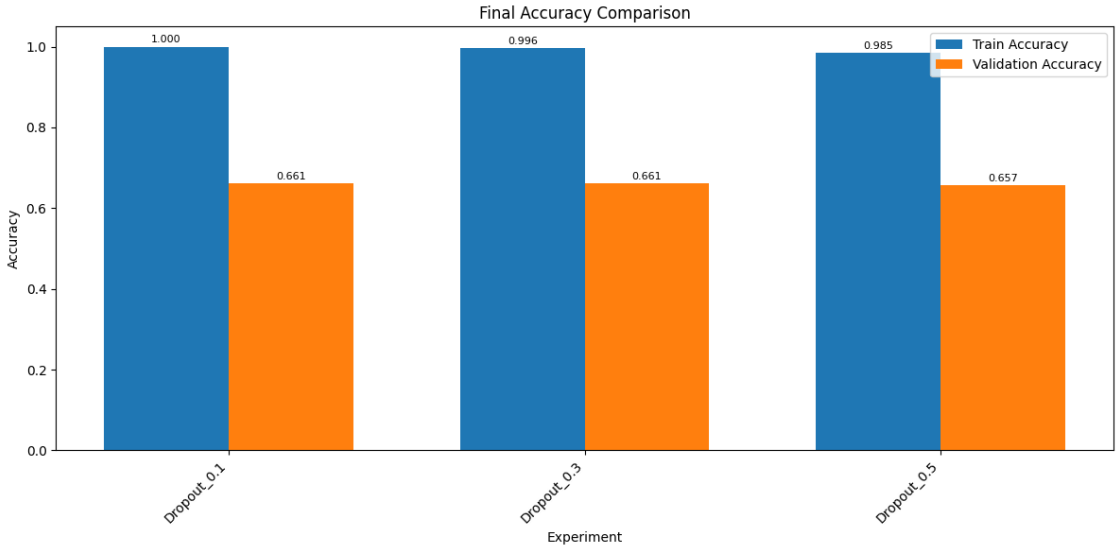
[ ]:
```
print("\nGraficando resultados del ajuste de Dropout...")
plot_experiment_results(dropout_tuning_results, metric='accuracy_val_vs_train')
plot_experiment_results(dropout_tuning_results, metric='val_loss')
compare_final_metrics(dropout_tuning_results)

best_dropout_tuned_model = max(dropout_tuning_results, key=lambda x: x.
 ↪get('final_val_accuracy', -float('inf')))
print(f"\n--- Mejor Modelo Después del Ajuste de Dropout ---")
print(f"Nombre: {best_dropout_tuned_model.get('name', 'N/A')}")
print(f"Precisión de Validación: {best_dropout_tuned_model.
 ↪get('final_val_accuracy', 'N/A'):.4f}")
```

```python
print(f"Dropout Rate: {best_dropout_tuned_model.get('dropout_rate', 'N/A')}")
print(f"Configuración Completa:")
for key, value in best_dropout_tuned_model.items():
    if key not in ['history']:
        print(f"  {key}: {value}")
```

Graficando resultados del ajuste de Dropout…








```
--- Mejor Modelo Después del Ajuste de Dropout ---
Nombre: Dropout_0.1
Precisión de Validación: 0.6608
Dropout Rate: 0.1
```

```
Configuración Completa:
  learning_rate: 0.001
  l2_lambda: 0.0001
  dropout_rate: 0.1
  use_batch_norm: False
  epochs: 4000
  batch_size: 64
  optimizer: adam
  lr_schedule: exponential
  early_stopping_patience: 15
  name: Dropout_0.1
  model: <src.improved_neural_network.ImprovedNeuralNetwork object at
0x70d0fd5f4f20>
  final_train_loss: 0.02018109722402314
  final_train_accuracy: 0.999687206756334
  final_val_loss: 1.65058578055422
  final_val_accuracy: 0.6608478802992519
  training_time: 11.681177139282227
```

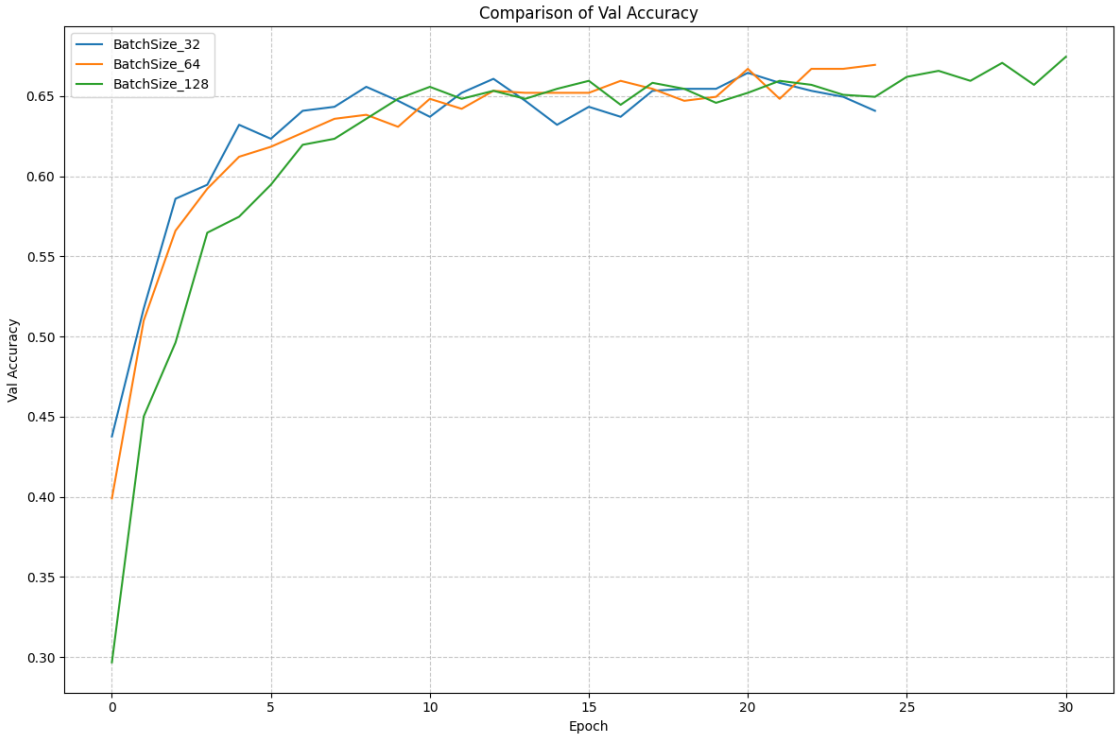
[ ]:
```
batch_sizes_to_test = [32, 64, 128]
batch_size_tuning_experiments = []

best_dropout = best_dropout_tuned_model['dropout_rate']

config['dropout_rate'] = best_dropout

for bs_value in batch_sizes_to_test:
    batch_size_tuning_experiments.append(config.copy())
    batch_size_tuning_experiments[-1]['batch_size'] = bs_value
    batch_size_tuning_experiments[-1]['name'] = f"BatchSize_{bs_value}"

batch_size_tuning_results = run_experiments(
    X_train, y_train, X_val, y_val,
    ImprovedNeuralNetwork,
    best_arch_result['layer_sizes'],
    batch_size_tuning_experiments
)
```

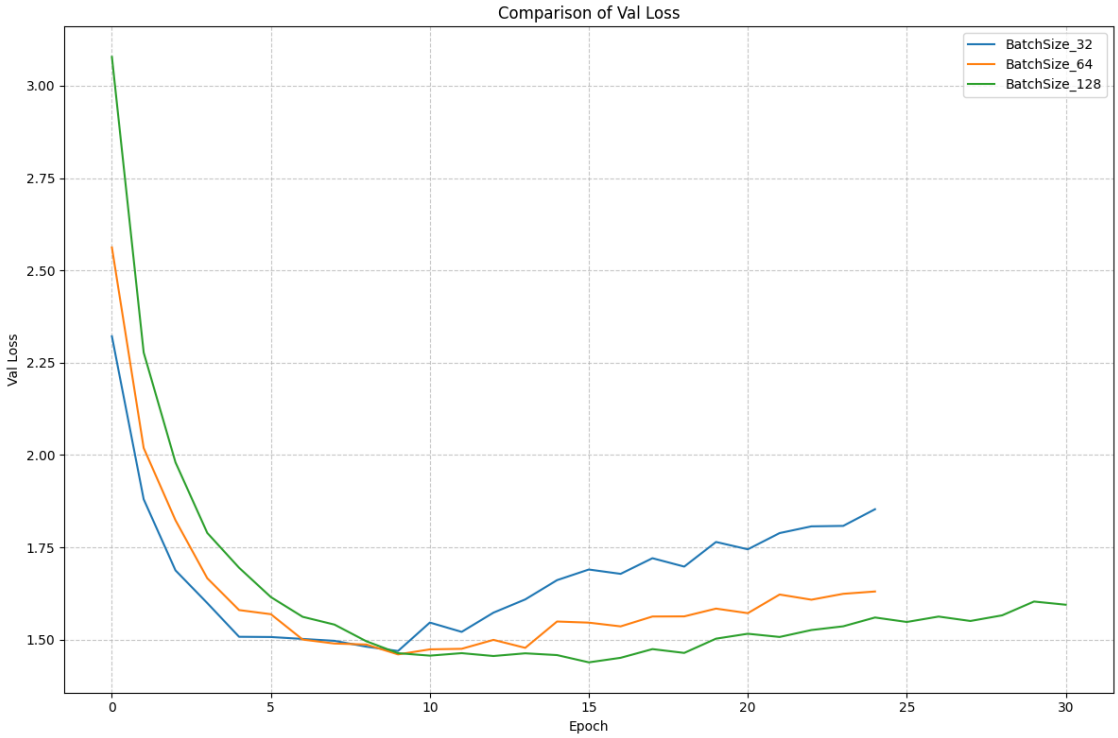
```
Running experiment: BatchSize_32
Epoch: [>                                                  ] 0% - train_loss:
0.0131 - train_acc: 0.9997 - val_loss: 1.8533 - val_acc: 0.6409 - lr: 0.0010
Early stopping triggered at epoch 25

Best result for BatchSize_32: val_loss = 1.4695, val_accuracy = 0.6471

Running experiment: BatchSize_64
Epoch: [>                                                  ] 0% - train_loss:
```

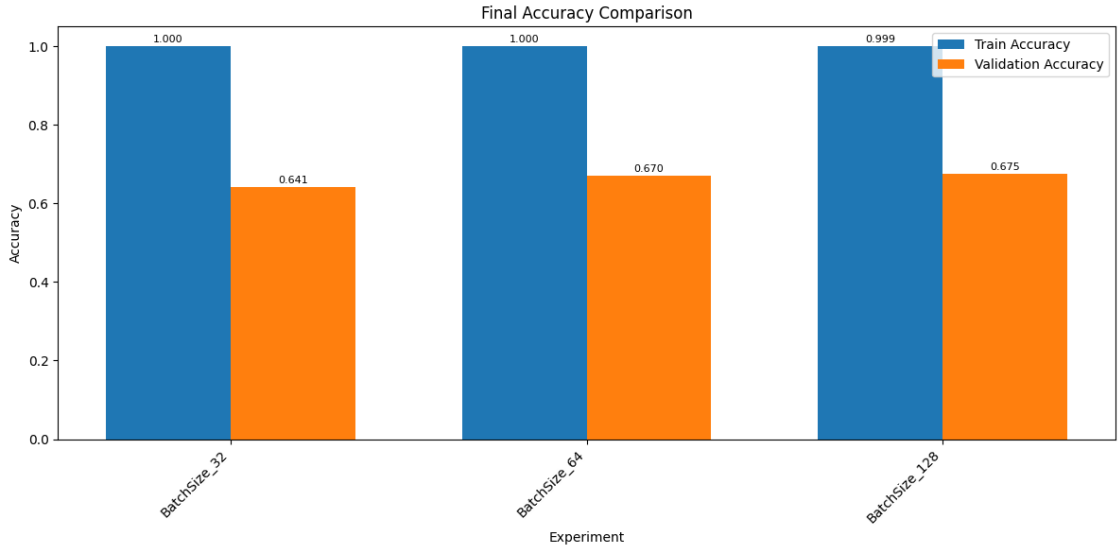
```
0.0248 - train_acc: 1.0000 - val_loss: 1.6306 - val_acc: 0.6696 - lr: 0.0010
Early stopping triggered at epoch 25

Best result for BatchSize_64: val_loss = 1.4602, val_accuracy = 0.6309

Running experiment: BatchSize_128
Epoch: [>                                                  ] 0% - train_loss:
0.0304 - train_acc: 0.9994 - val_loss: 1.5948 - val_acc: 0.6746 - lr: 0.0010
Early stopping triggered at epoch 31

Best result for BatchSize_128: val_loss = 1.4388, val_accuracy = 0.6596
```

[ ]:
```
print("\nGraficando resultados del ajuste de Batch Size...")
plot_experiment_results(batch_size_tuning_results, metric='val_accuracy')
plot_experiment_results(batch_size_tuning_results, metric='val_loss')
compare_final_metrics(batch_size_tuning_results)

best_bs_tuned_model = max(batch_size_tuning_results, key=lambda x: x.
 ↪get('final_val_accuracy', -float('inf')))
print(f"\n--- Mejor Modelo Después del Ajuste de Batch Size ---")
print(f"Nombre: {best_bs_tuned_model.get('name', 'N/A')}")
print(f"Precisión de Validación: {best_bs_tuned_model.get('final_val_accuracy',␣
 ↪'N/A'):.4f}")
print(f"Batch Size: {best_bs_tuned_model.get('batch_size', 'N/A')}")
print(f"Configuración Completa:")
for key, value in best_bs_tuned_model.items():
    if key not in ['history']:
        print(f"  {key}: {value}")
```

```
Graficando resultados del ajuste de Batch Size…
```


Comparison of Val Accuracy
BatchSize_32
BatchSize_64
BatchSize_128
Val Accuracy
Epoch
0.30
0.35
0.40
0.45
0.50
0.55
0.60
0.65
0
5
10
15
20
25
30


Comparison of Val Loss
BatchSize_32
BatchSize_64
BatchSize_128
Val Loss
Epoch
1.50
1.75
2.00
2.25
2.50
2.75
3.00
0
5
10
15
20
25
30




```
--- Mejor Modelo Después del Ajuste de Batch Size ---
Nombre: BatchSize_128
Precisión de Validación: 0.6746
Batch Size: 128
Configuración Completa:
  learning_rate: 0.001
  l2_lambda: 0.0001
  dropout_rate: 0.1
  use_batch_norm: False
  epochs: 4000
  batch_size: 128
  optimizer: adam
  lr_schedule: exponential
  early_stopping_patience: 15
  name: BatchSize_128
  model: <src.improved_neural_network.ImprovedNeuralNetwork object at
0x70d0fd8b4c50>
  final_train_loss: 0.03041992524748165
  final_train_accuracy: 0.9993744135126681
  final_val_loss: 1.5947655359172872
  final_val_accuracy: 0.6745635910224439
  training_time: 9.439747333526611
```

[ ]:
```
schedules_to_test = ['exponential', 'linear', None]

schedule_tuning_experiments = []

best_bs = best_bs_tuned_model['batch_size']
```

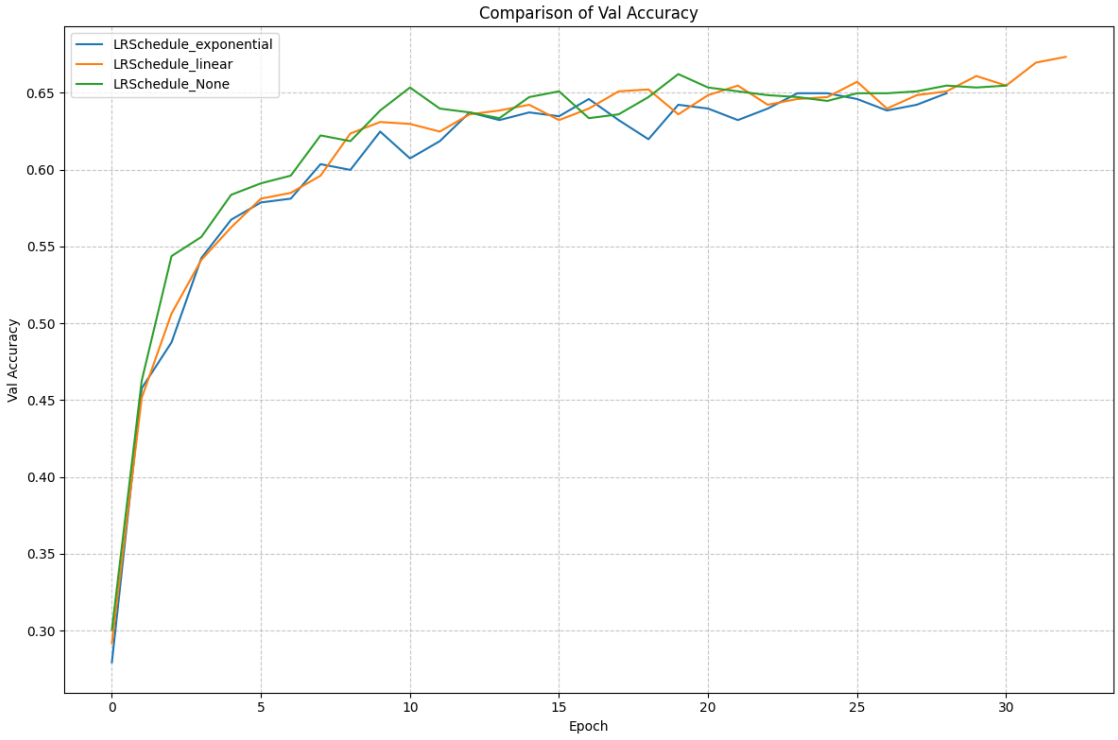
```
config['batch_size'] = best_bs

for sched in schedules_to_test:
    schedule_tuning_experiments.append(config.copy())
    schedule_tuning_experiments[-1]['lr_schedule'] = sched
    schedule_tuning_experiments[-1]['name'] = f"LRSchedule_{sched}"

schedule_tuning_results = run_experiments(
    X_train, y_train, X_val, y_val,
    ImprovedNeuralNetwork,
    best_arch_result['layer_sizes'],
    schedule_tuning_experiments
)
```

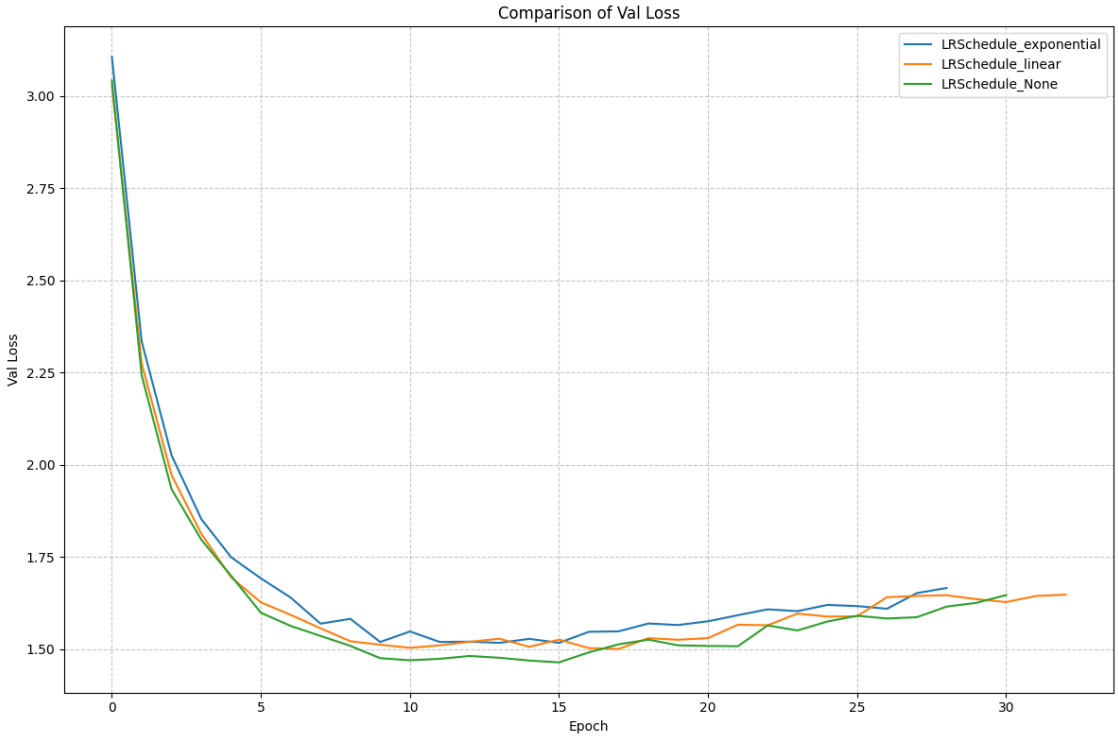
```
Running experiment: LRSchedule_exponential
Epoch: [>                                                  ] 0% - train_loss:
0.0428 - train_acc: 0.9991 - val_loss: 1.6663 - val_acc: 0.6496 - lr: 0.0010
Early stopping triggered at epoch 29

Best result for LRSchedule_exponential: val_loss = 1.5176, val_accuracy = 0.6322

Running experiment: LRSchedule_linear
Epoch: [>                                                  ] 0% - train_loss:
0.0278 - train_acc: 1.0000 - val_loss: 1.6482 - val_acc: 0.6733 - lr: 0.0010
Early stopping triggered at epoch 33

Best result for LRSchedule_linear: val_loss = 1.5010, val_accuracy = 0.6509

Running experiment: LRSchedule_None
Epoch: [>                                                  ] 0% - train_loss:
0.0336 - train_acc: 0.9991 - val_loss: 1.6471 - val_acc: 0.6546 - lr: 0.0010
Early stopping triggered at epoch 31

Best result for LRSchedule_None: val_loss = 1.4648, val_accuracy = 0.6509
```

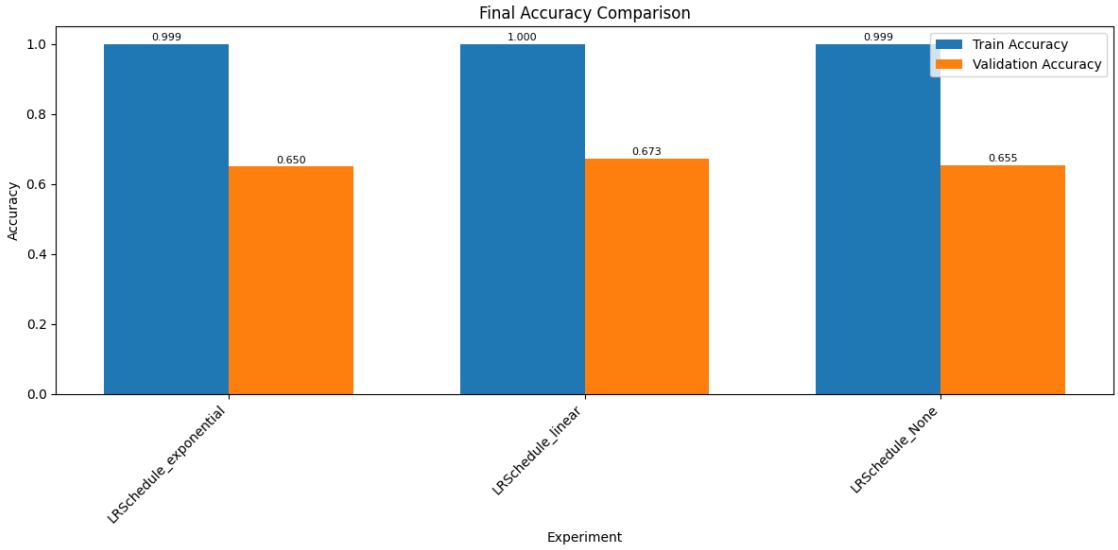
[ ]:
```
print("\nGraficando resultados del ajuste de LR Schedule...")
plot_experiment_results(schedule_tuning_results, metric='val_accuracy')
plot_experiment_results(schedule_tuning_results, metric='val_loss')
compare_final_metrics(schedule_tuning_results)

best_sched_tuned_model = max(schedule_tuning_results, key=lambda x: x.
 ↪get('final_val_accuracy', -float('inf')))
print(f"\n--- Mejor Modelo Después del Ajuste de LR Schedule ---")
print(f"Nombre: {best_sched_tuned_model.get('name', 'N/A')}")
```

```
print(f"Precisión de Validación: {best_sched_tuned_model.
 ↪get('final_val_accuracy', 'N/A'):.4f}")
print(f"Schedule: {best_sched_tuned_model.get('lr_schedule', 'N/A')}")
print(f"Configuración Completa:")
for key, value in best_sched_tuned_model.items():
    if key not in ['history']:
        print(f"  {key}: {value}")
```

```
Graficando resultados del ajuste de LR Schedule…
```








```
--- Mejor Modelo Después del Ajuste de LR Schedule ---
Nombre: LRSchedule_linear
Precisión de Validación: 0.6733
Schedule: linear
```

```
Configuración Completa:
  learning_rate: 0.001
  l2_lambda: 0.0001
  dropout_rate: 0.1
  use_batch_norm: False
  epochs: 4000
  batch_size: 128
  optimizer: adam
  lr_schedule: linear
  early_stopping_patience: 15
  name: LRSchedule_linear
  model: <src.improved_neural_network.ImprovedNeuralNetwork object at
0x70d0fd87e600>
  final_train_loss: 0.02781352558270429
  final_train_accuracy: 1.0
  final_val_loss: 1.648249769922454
  final_val_accuracy: 0.6733167082294265
  training_time: 10.584905862808228
```

[ ]:
```python
final_model = best_sched_tuned_model['model']
y_pred_test = final_model.forward(X_test, training=False)
test_loss = final_model.cross_entropy_loss(y_test, y_pred_test)
test_accuracy = final_model.accuracy(y_test, y_pred_test)

print(f"\nFinal model performance on test set:")
print(f"Test loss: {test_loss:.4f}, test accuracy: {test_accuracy:.4f}")

conf_matrix = final_model.confusion_matrix(y_test, y_pred_test)

model_M1 = final_model
save_model(model_M1, "model_M1.pkl")
```

```
Final model performance on test set:
Test loss: 1.5488, test accuracy: 0.6414
Model successfully saved to modelos/model_M1.pkl
```

0.1.4 4. Desarrollo de una Red Neuronal con PyTorch

- Utilizando PyTorch, entrenar una red neuronal con la arquitectura y los hiperparámetros hallados en el ejercicio anterior. Llmaremos a este modelo M2.
- Utilizando PyTorch, explorar cambios en la cantidad de capas ocultas y unidades ocultas por capa, y determinar la configuración que funcione mejor. LLamaremos a este modelo M3.
- Utilizando PyTorch, encontrar una arquitectura (capas ocultas y unidades por capa) que produzca overfitting. Llamaremos a este modelo M4. Comparar la performance sobre el conjunto de test de los siguientes cuatro modelos:
  1. El modelo base de implementación propia (M0).
  2. La mejor arquitectura obtenida con la implementación propia (M1).

3. Modelo en PyTorch, usando la misma arquiterctura e hiperparámetros que en la implementación propia (M2).
4. La mejor arquitectura obtenida en PyTorch (M3).
5. Una arquitectura en PyTorch con sobreajuste (M4).

[ ]:
```
print("\n4.a) Cargando el modelo base (M0) de implementación propia...")
model_M0 = load_model("model_M0.pkl")

test_acc_M0, test_loss_M0, test_pred_M0 = model_M0.evaluate(X_test, y_test)
print(f"Modelo M0 en test: Accuracy={test_acc_M0:.4f}, Loss={test_loss_M0:.4f}")
```

```
4.a) Cargando el modelo base (M0) de implementación propia…
Model successfully loaded from modelos/model_M0.pkl
Modelo M0 en test: Accuracy=0.5814, Loss=1.8024
```

[ ]:
```
print("\n4.b) Cargando la mejor arquitectura de implementación propia (M1)...")
model_M1 = load_model("model_M1.pkl")

test_acc_M1, test_loss_M1, test_pred_M1 = model_M1.evaluate(X_test, y_test)
print(f"Modelo M1 en test: Accuracy={test_acc_M1:.4f}, Loss={test_loss_M1:.4f}")
```

```
4.b) Cargando la mejor arquitectura de implementación propia (M1)…
Model successfully loaded from modelos/model_M1.pkl
Modelo M1 en test: Accuracy=0.5784, Loss=1.8049
```

[ ]:
```
print("\n4.c) Implementando modelo en PyTorch con misma arquitectura (M2)...")
model_M2 = PyTorchNetwork(
    layer_sizes=layer_sizes,
    learning_rate=0.001,
    l2_lambda=0.0005,
    dropout_rate=0.2
)

print("Entrenando modelo M2...")
history_M2 = model_M2.train_model(
    X_train, y_train, X_val, y_val,
    epochs=1000, batch_size=64,
    early_stopping_patience=10,
    verbose=1
)

save_model(model_M2, "model_M2.pkl")

test_acc_M2, test_loss_M2, test_pred_M2 = model_M2.evaluate(X_test, y_test)
print(f"Modelo M2 en test: Accuracy={test_acc_M2:.4f}, Loss={test_loss_M2:.4f}")
```

```
4.c) Implementando modelo en PyTorch con misma arquitectura (M2)…
Entrenando modelo M2…
Epoch: [=>                                                    ] 3% - train_loss:
0.6258 - train_acc: 0.8205 - val_loss: 1.5350 - val_acc: 0.6247
Early stopping triggered at epoch 36

Model successfully saved to modelos/model_M2.pkl
Modelo M2 en test: Accuracy=0.6164, Loss=1.6651
```

[ ]:
```python
print("\n4.d) Explorando diferentes arquitecturas con PyTorch...")

architectures = [
    {'name': 'One Hidden Layer (200)', 'layers': [input_size, 200,␣
 ↪output_size]},
    {'name': 'Two Hidden Layers (100, 80)', 'layers': [input_size, 100, 80,␣
 ↪output_size]},
    {'name': 'Three Hidden Layers (120, 80, 60)', 'layers': [input_size, 120,␣
 ↪80, 60, output_size]},
    {'name': 'Wide Network (200, 150)', 'layers': [input_size, 200, 150,␣
 ↪output_size]},
    {'name': 'Deep Network (100, 80, 60, 40)', 'layers': [input_size, 100, 80,␣
 ↪60, 40, output_size]}
]

architecture_results = []

for arch in architectures:
    print(f"\nEntrenando {arch['name']}...")

    model = PyTorchNetwork(
        layer_sizes=arch['layers'],
        learning_rate=0.001,
        l2_lambda=0.0005,
        dropout_rate=0.2
    )

    history = model.train_model(
        X_train, y_train, X_val, y_val,
        epochs=50, batch_size=64,
        early_stopping_patience=10,
        verbose=1
    )

    test_acc, test_loss, _ = model.evaluate(X_test, y_test)

    arch_result = arch.copy()
```

```
    arch_result.update({
        'model': model,
        'history': history,
        'test_accuracy': test_acc,
        'test_loss': test_loss
    })

    architecture_results.append(arch_result)
    print(f"Arquitectura {arch['name']} en test: Accuracy={test_acc:.4f},␣
↪Loss={test_loss:.4f}")

best_arch = max(architecture_results, key=lambda x: x['test_accuracy'])
print(f"\nMejor arquitectura: {best_arch['name']}")
print(f"Test accuracy: {best_arch['test_accuracy']:.4f}")

model_M3 = best_arch['model']
test_acc_M3 = best_arch['test_accuracy']
test_loss_M3 = best_arch['test_loss']

save_model(model_M3, "model_M3.pkl")
```

```
4.d) Explorando diferentes arquitecturas con PyTorch…

Entrenando One Hidden Layer (200)…
Epoch: [==========================>                      ] 56% - train_loss:
0.3119 - train_acc: 0.9343 - val_loss: 1.4980 - val_acc: 0.6446
Early stopping triggered at epoch 28

Arquitectura One Hidden Layer (200) en test: Accuracy=0.6374, Loss=1.5968

Entrenando Two Hidden Layers (100, 80)…
Epoch: [================================>                 ] 66% - train_loss:
0.6970 - train_acc: 0.8029 - val_loss: 1.5538 - val_acc: 0.6209
Early stopping triggered at epoch 33

Arquitectura Two Hidden Layers (100, 80) en test: Accuracy=0.6204, Loss=1.6679

Entrenando Three Hidden Layers (120, 80, 60)…
Epoch: [============================>                    ] 60% - train_loss:
0.8226 - train_acc: 0.7535 - val_loss: 1.6741 - val_acc: 0.5885
Early stopping triggered at epoch 30

Arquitectura Three Hidden Layers (120, 80, 60) en test: Accuracy=0.5844,
Loss=1.7637

Entrenando Wide Network (200, 150)…
```

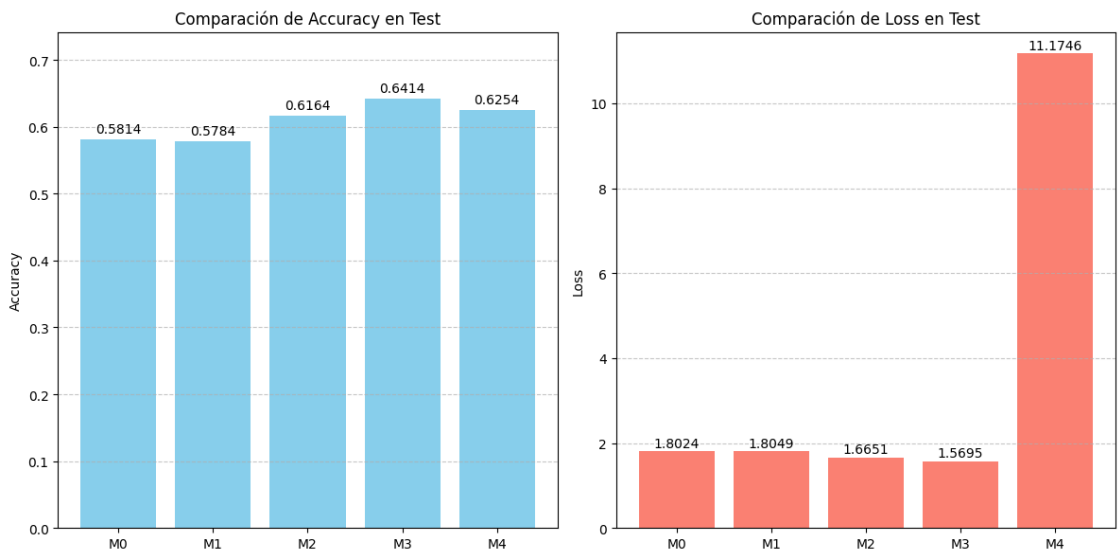
```
Epoch: [========================>                          ] 50% - train_loss:
0.3606 - train_acc: 0.9030 - val_loss: 1.4727 - val_acc: 0.6546
Early stopping triggered at epoch 25

Arquitectura Wide Network (200, 150) en test: Accuracy=0.6414, Loss=1.5695

Entrenando Deep Network (100, 80, 60, 40)…
Epoch: [==============================>                    ] 62% - train_loss:
1.2136 - train_acc: 0.6309 - val_loss: 2.1203 - val_acc: 0.5112
Early stopping triggered at epoch 31

Arquitectura Deep Network (100, 80, 60, 40) en test: Accuracy=0.4975,
Loss=2.3177

Mejor arquitectura: Wide Network (200, 150)
Test accuracy: 0.6414
Model successfully saved to modelos/model_M3.pkl
```

[ ]:
```python
print("\n4.e) Implementando arquitectura con overfitting...")

model_M4 = PyTorchNetwork(
    layer_sizes=[input_size, 300, 200, 150, output_size],
    learning_rate=0.001,
    l2_lambda=0.0,
    dropout_rate=0.0
)

print("Entrenando modelo M4 (overfitting)...")
history_M4 = model_M4.train_model(
    X_train, y_train, X_val, y_val,
    epochs=2000, batch_size=64,
    early_stopping_patience=None,
    verbose=1
)

save_model(model_M4, "model_M4.pkl")

train_acc_M4, train_loss_M4, _ = model_M4.evaluate(X_train, y_train)
test_acc_M4, test_loss_M4, test_pred_M4 = model_M4.evaluate(X_test, y_test)

print(f"Modelo M4 (overfitting) en train: Accuracy={train_acc_M4:.4f},␣
↪Loss={train_loss_M4:.4f}")
print(f"Modelo M4 (overfitting) en test: Accuracy={test_acc_M4:.4f},␣
↪Loss={test_loss_M4:.4f}")
print(f"Diferencia entre train y test accuracy: {train_acc_M4 - test_acc_M4:.
↪4f}")
```

```
4.e) Implementando arquitectura con overfitting…
Entrenando modelo M4 (overfitting)…
Epoch: [==================================================>] 100% - train_loss:
0.0000 - train_acc: 1.0000 - val_loss: 10.4639 - val_acc: 0.6421
Model successfully saved to modelos/model_M4.pkl
Modelo M4 (overfitting) en train: Accuracy=1.0000, Loss=0.0000
Modelo M4 (overfitting) en test: Accuracy=0.6254, Loss=11.1746
Diferencia entre train y test accuracy: 0.3746
```

[ ]:
```
models_comparison = {
    'M0': {'name': 'Modelo base implementación propia', 'test_acc':␣
 ↪test_acc_M0, 'test_loss': test_loss_M0},
    'M1': {'name': 'Mejor modelo implementación propia', 'test_acc':␣
 ↪test_acc_M1, 'test_loss': test_loss_M1},
    'M2': {'name': 'PyTorch misma arquitectura que M1', 'test_acc':␣
 ↪test_acc_M2, 'test_loss': test_loss_M2},
    'M3': {'name': f'Mejor arquitectura PyTorch ({best_arch["name"]})',␣
 ↪'test_acc': test_acc_M3, 'test_loss': test_loss_M3},
    'M4': {'name': 'PyTorch con overfitting', 'test_acc': test_acc_M4,␣
 ↪'test_loss': test_loss_M4}
}

plt.figure(figsize=(12, 6))
models = list(models_comparison.keys())
accuracies = [models_comparison[m]['test_acc'] for m in models]
losses = [models_comparison[m]['test_loss'] for m in models]

plt.subplot(1, 2, 1)
bars = plt.bar(models, accuracies, color='skyblue')

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{acc:.4f}', ha='center', fontsize=10)

plt.title('Comparación de Accuracy en Test')
plt.ylabel('Accuracy')
plt.ylim(0, max(accuracies) + 0.1)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.subplot(1, 2, 2)
bars = plt.bar(models, losses, color='salmon')

for bar, loss in zip(bars, losses):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.1,
             f'{loss:.4f}', ha='center', fontsize=10)

plt.title('Comparación de Loss en Test')
```

```
plt.ylabel('Loss')
plt.ylim(0, max(losses) + 0.5)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```




```
Tabla comparativa de modelos:
================================================================================
Modelo Descripción                                Test Accuracy   Test Loss
--------------------------------------------------------------------------------
M0     Modelo base implementación propia         0.5814          1.8024
M1     Mejor modelo implementación propia        0.5784          1.8049
M2     PyTorch misma arquitectura que M1         0.6164          1.6651
M3     Mejor arquitectura PyTorch (Wide Network (200, 150)) 0.6414
1.5695
M4     PyTorch con overfitting                   0.6254          11.1746
================================================================================

El mejor modelo global es: M3 - Mejor arquitectura PyTorch (Wide Network (200,
150))
Test accuracy: 0.6414
```

[ ]:
```
print("\nTabla comparativa de modelos:")
print("=" * 80)
print(f"{'Modelo':<5} {'Descripción':<40} {'Test Accuracy':<15} {'Test Loss':
↪<15}")
print("-" * 80)
```

```
for model_id, model_info in models_comparison.items():
    print(f"{model_id:<5} {model_info['name']:<40} {model_info['test_acc']:<15.
 ↪4f} {model_info['test_loss']:<15.4f}")
print("=" * 80)

best_model_id = max(models_comparison.keys(), key=lambda m:␣
 ↪models_comparison[m]['test_acc'])
print(f"\nEl mejor modelo global es: {best_model_id} -␣
 ↪{models_comparison[best_model_id]['name']}")
print(f"Test accuracy: {models_comparison[best_model_id]['test_acc']:.4f}")
```

0.1.5 5. Desafío

- Utilizando el modelo que considere que sea el mejor, predecir las probabilidades a-posteriori de cada clase del dataset X_COMP.npy y generar un archivo .csv con las predicciones llamado Apellido_Nombre_predicciones.csv. El archivo predicciones.csv debe tener una fila por muestra y las columnas deben ser las probabilidades a posteriori de cada clase.

[ ]:
```
model = PyTorchNetwork(
    layer_sizes=best_arch['layers'],
    learning_rate=0.001,
    l2_lambda=0.0005,
    dropout_rate=0.2
)

X_train, _, X_test, y_train, _, y_test = stratified_split(
    X_images / 255.0 , y_images, train_ratio=0.8, val_ratio=0.0, test_ratio=0.
 ↪2, random_state=42
)

history = model.train_model(
    X_train, y_train, X_test, y_test,
    epochs=50, batch_size=64,
    early_stopping_patience=10,
    verbose=2
)

save_model(model, "model_final.pkl")
```

```
Epoch: [=======================>                          ] 50% - train_loss:
0.3653 - train_acc: 0.9002 - val_loss: 1.4300 - val_acc: 0.6713
Early stopping triggered at epoch 25


Training completed in 4.94 seconds
Final train loss: 0.3653, train accuracy: 0.9002
Final val loss: 1.4300, val accuracy: 0.6713
```

```
Best epoch: 15
Best val loss: 1.3744, best val accuracy: 0.6533
Model successfully saved to modelos/model_final.pkl
```

[ ]:
```python
X_COMP = np.load("data/X_COMP.npy")
X_COMP = X_COMP / 255.0

model = load_model("model_final.pkl")

probabilidades = model.forward(X_COMP)

if isinstance(probabilidades, torch.Tensor):
    probabilidades = probabilidades.detach().numpy()

columnas = [f"Clase_{i}" for i in range(probabilidades.shape[1])]

df_predicciones = pd.DataFrame(probabilidades, columns=columnas)
df_predicciones.to_csv("Ostrovsky_Eliana_predicciones.csv", index=False)
```

```
Model successfully loaded from modelos/model_final.pkl
```